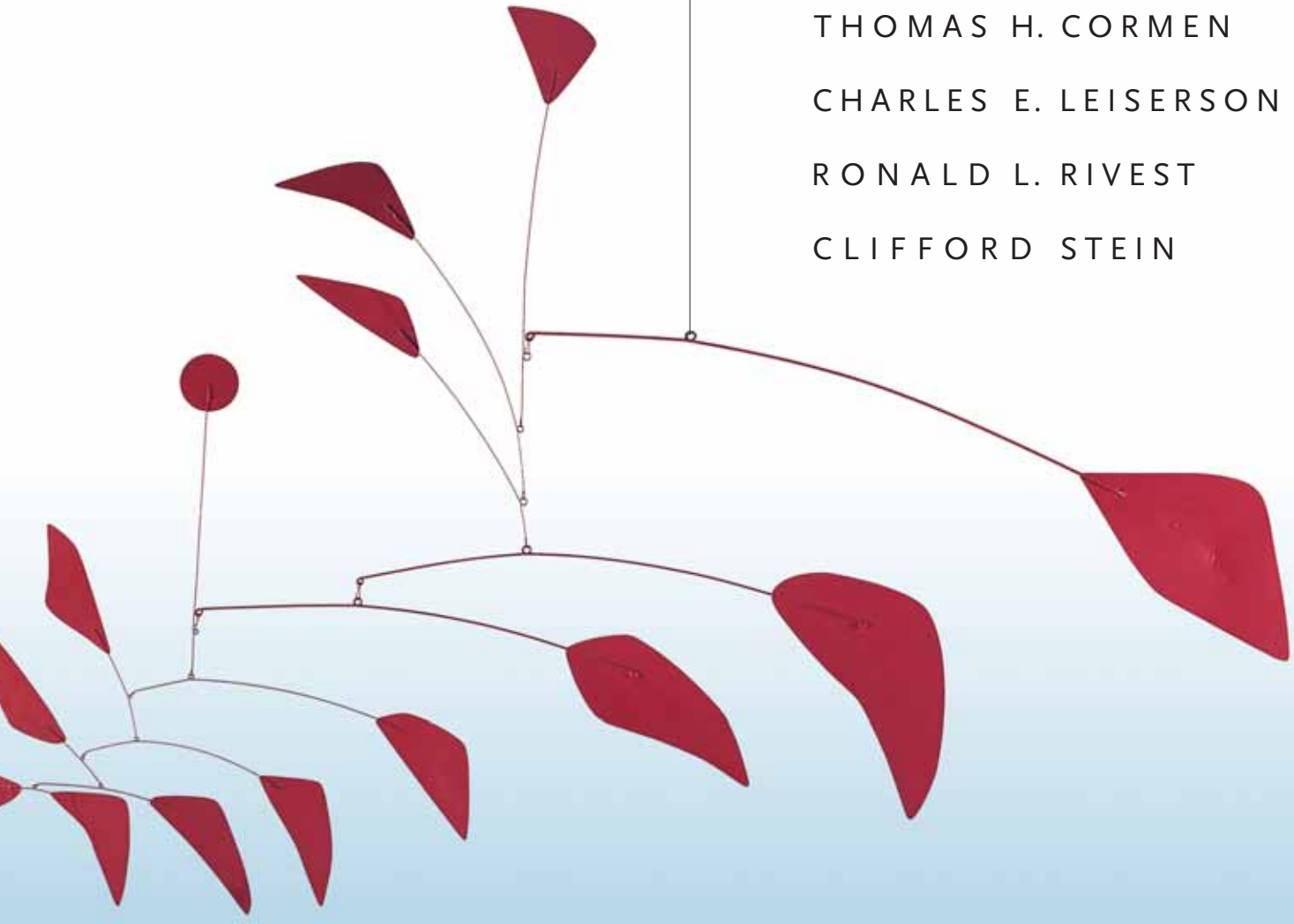
THOMAS H. CORMEN
CHARLES E. LEISERSON
RONALD L. RIVEST
CLIFFORD STEIN



INTRODUCTION TO ALGORITHMS

THIRD EDITION

Thomas H. Cormen
Charles E. Leiserson
Ronald L. Rivest
Clifford Stein

Introduction to Algorithms

Third Edition

The MIT Press
Cambridge, Massachusetts London, England

© 2009 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form or by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

For information about special quantity discounts, please email special_sales@mitpress.mit.edu.

This book was set in Times Roman and Mathtime Pro 2 by the authors.

Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data

Introduction to algorithms / Thomas H. Cormen . . . [et al.].—3rd ed.
p. cm.
Includes bibliographical references and index.
ISBN 978-0-262-03384-8 (hardcover : alk. paper)—ISBN 978-0-262-53305-8 (pbk. : alk. paper)
1. Computer programming. 2. Computer algorithms. I. Cormen, Thomas H.

QA76.6.I5858 2009
005.1—dc22

2009008593

10 9 8 7 6 5 4 3

Contents

Basic Sorting Algorithms

Source and supplements

1 Principles of sorting *5*

Original reading 5-25

Explanations, figures, proofs, and Q&A 26-33

2 Bubble sort *34*

Original reading 34-35

Explanations, figures, proofs, and Q&A 36-44

3 Selection sort *45*

Original reading 45-45

Explanations, figures, proofs, and Q&A 46-53

4 Insertion sort *54*

Original reading 54-66

Explanations, figures, proofs, and Q&A 67-71

5 Merge sort *72*

Original reading 72-82

Explanations, figures, proofs, and Q&A 83-89

6 Counting sort *90*

Original reading 90-93

Explanations, figures, proofs, and Q&A 94-100

Contents numbers refer to page positions in this 100-page PDF.
Original printed pagination and original page layouts are preserved.
Pages labeled Supplement are new material, not original CLRS pages.
Detailed supplemental sections are available in the PDF bookmarks.
Whole source pages may include brief adjoining topics and references.

1 The Role of Algorithms in Computing

What are algorithms? Why is the study of algorithms worthwhile? What is the role of algorithms relative to other technologies used in computers? In this chapter, we will answer these questions.

1.1 Algorithms

Informally, an ***algorithm*** is any well-defined computational procedure that takes some value, or set of values, as ***input*** and produces some value, or set of values, as ***output***. An algorithm is thus a sequence of computational steps that transform the input into the output.

We can also view an algorithm as a tool for solving a well-specified ***computational problem***. The statement of the problem specifies in general terms the desired input/output relationship. The algorithm describes a specific computational procedure for achieving that input/output relationship.

For example, we might need to sort a sequence of numbers into nondecreasing order. This problem arises frequently in practice and provides fertile ground for introducing many standard design techniques and analysis tools. Here is how we formally define the ***sorting problem***:

Input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

For example, given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$. Such an input sequence is called an ***instance*** of the sorting problem. In general, an ***instance of a problem*** consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.

Because many programs use it as an intermediate step, sorting is a fundamental operation in computer science. As a result, we have a large number of good sorting algorithms at our disposal. Which algorithm is best for a given application depends on—among other factors—the number of items to be sorted, the extent to which the items are already somewhat sorted, possible restrictions on the item values, the architecture of the computer, and the kind of storage devices to be used: main memory, disks, or even tapes.

An algorithm is said to be ***correct*** if, for every input instance, it halts with the correct output. We say that a correct algorithm ***solves*** the given computational problem. An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer. Contrary to what you might expect, incorrect algorithms can sometimes be useful, if we can control their error rate. We shall see an example of an algorithm with a controllable error rate in Chapter 31 when we study algorithms for finding large prime numbers. Ordinarily, however, we shall be concerned only with correct algorithms.

An algorithm can be specified in English, as a computer program, or even as a hardware design. The only requirement is that the specification must provide a precise description of the computational procedure to be followed.

What kinds of problems are solved by algorithms?

Sorting is by no means the only computational problem for which algorithms have been developed. (You probably suspected as much when you saw the size of this book.) Practical applications of algorithms are ubiquitous and include the following examples:

- The Human Genome Project has made great progress toward the goals of identifying all the 100,000 genes in human DNA, determining the sequences of the 3 billion chemical base pairs that make up human DNA, storing this information in databases, and developing tools for data analysis. Each of these steps requires sophisticated algorithms. Although the solutions to the various problems involved are beyond the scope of this book, many methods to solve these biological problems use ideas from several of the chapters in this book, thereby enabling scientists to accomplish tasks while using resources efficiently. The savings are in time, both human and machine, and in money, as more information can be extracted from laboratory techniques.
- The Internet enables people all around the world to quickly access and retrieve large amounts of information. With the aid of clever algorithms, sites on the Internet are able to manage and manipulate this large volume of data. Examples of problems that make essential use of algorithms include finding good routes on which the data will travel (techniques for solving such problems appear in

Efficiency

Different algorithms devised to solve the same problem often differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.

As an example, in Chapter 2, we will see two algorithms for sorting. The first, known as ***insertion sort***, takes time roughly equal to $c_1 n^2$ to sort n items, where c_1 is a constant that does not depend on n. That is, it takes time roughly proportional to n^2. The second, ***merge sort***, takes time roughly equal to $c_2 n \lg n$, where $\lg n$ stands for $\log_2 n$ and c_2 is another constant that also does not depend on n. Insertion sort typically has a smaller constant factor than merge sort, so that $c_1 < c_2$. We shall see that the constant factors can have far less of an impact on the running time than the dependence on the input size n. Let's write insertion sort's running time as $c_1 n \cdot n$ and merge sort's running time as $c_2 n \cdot \lg n$. Then we see that where insertion sort has a factor of n in its running time, merge sort has a factor of $\lg n$, which is much smaller. (For example, when $n = 1000$, $\lg n$ is approximately 10, and when n equals one million, $\lg n$ is approximately only 20.) Although insertion sort usually runs faster than merge sort for small input sizes, once the input size n becomes large enough, merge sort's advantage of $\lg n$ vs. n will more than compensate for the difference in constant factors. No matter how much smaller c_1 is than c_2, there will always be a crossover point beyond which merge sort is faster.

For a concrete example, let us pit a faster computer (computer A) running insertion sort against a slower computer (computer B) running merge sort. They each must sort an array of 10 million numbers. (Although 10 million numbers might seem like a lot, if the numbers are eight-byte integers, then the input occupies about 80 megabytes, which fits in the memory of even an inexpensive laptop computer many times over.) Suppose that computer A executes 10 billion instructions per second (faster than any single sequential computer at the time of this writing) and computer B executes only 10 million instructions per second, so that computer A is 1000 times faster than computer B in raw computing power. To make the difference even more dramatic, suppose that the world's craftiest programmer codes insertion sort in machine language for computer A, and the resulting code requires $2n^2$ instructions to sort n numbers. Suppose further that just an average programmer implements merge sort, using a high-level language with an inefficient compiler, with the resulting code taking $50n \lg n$ instructions. To sort 10 million numbers, computer A takes

$$\frac{2 \cdot (10^7)^2 \text{ instructions}}{10^{10} \text{ instructions/second}} = 20{,}000 \text{ seconds (more than 5.5 hours)} ,$$

while computer B takes

$$\frac{50 \cdot 10^7 \lg 10^7 \text{ instructions}}{10^7 \text{ instructions/second}} \approx 1163 \text{ seconds (less than 20 minutes)} .$$

By using an algorithm whose running time grows more slowly, even with a poor compiler, computer B runs more than 17 times faster than computer A! The advantage of merge sort is even more pronounced when we sort 100 million numbers: where insertion sort takes more than 23 days, merge sort takes under four hours. In general, as the problem size increases, so does the relative advantage of merge sort.

Algorithms and other technologies

The example above shows that we should consider algorithms, like computer hardware, as a ***technology***. Total system performance depends on choosing efficient algorithms as much as on choosing fast hardware. Just as rapid advances are being made in other computer technologies, they are being made in algorithms as well.

You might wonder whether algorithms are truly that important on contemporary computers in light of other advanced technologies, such as

- advanced computer architectures and fabrication technologies,
- easy-to-use, intuitive, graphical user interfaces (GUIs),
- object-oriented systems,
- integrated Web technologies, and
- fast networking, both wired and wireless.

The answer is yes. Although some applications do not explicitly require algorithmic content at the application level (such as some simple, Web-based applications), many do. For example, consider a Web-based service that determines how to travel from one location to another. Its implementation would rely on fast hardware, a graphical user interface, wide-area networking, and also possibly on object orientation. However, it would also require algorithms for certain operations, such as finding routes (probably using a shortest-path algorithm), rendering maps, and interpolating addresses.

Moreover, even an application that does not require algorithmic content at the application level relies heavily upon algorithms. Does the application rely on fast hardware? The hardware design used algorithms. Does the application rely on graphical user interfaces? The design of any GUI relies on algorithms. Does the application rely on networking? Routing in networks relies heavily on algorithms. Was the application written in a language other than machine code? Then it was processed by a compiler, interpreter, or assembler, all of which make extensive use

Introduction

This part presents several algorithms that solve the following ***sorting problem***:

Input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

The input sequence is usually an n-element array, although it may be represented in some other fashion, such as a linked list.

The structure of the data

In practice, the numbers to be sorted are rarely isolated values. Each is usually part of a collection of data called a ***record***. Each record contains a ***key***, which is the value to be sorted. The remainder of the record consists of ***satellite data***, which are usually carried around with the key. In practice, when a sorting algorithm permutes the keys, it must permute the satellite data as well. If each record includes a large amount of satellite data, we often permute an array of pointers to the records rather than the records themselves in order to minimize data movement.

In a sense, it is these implementation details that distinguish an algorithm from a full-blown program. A sorting algorithm describes the *method* by which we determine the sorted order, regardless of whether we are sorting individual numbers or large records containing many bytes of satellite data. Thus, when focusing on the problem of sorting, we typically assume that the input consists only of numbers. Translating an algorithm for sorting numbers into a program for sorting records

is conceptually straightforward, although in a given engineering situation other subtleties may make the actual programming task a challenge.

Why sorting?

Many computer scientists consider sorting to be the most fundamental problem in the study of algorithms. There are several reasons:

- Sometimes an application inherently needs to sort information. For example, in order to prepare customer statements, banks need to sort checks by check number.
- Algorithms often use sorting as a key subroutine. For example, a program that renders graphical objects which are layered on top of each other might have to sort the objects according to an "above" relation so that it can draw these objects from bottom to top. We shall see numerous algorithms in this text that use sorting as a subroutine.
- We can draw from among a wide variety of sorting algorithms, and they employ a rich set of techniques. In fact, many important techniques used throughout algorithm design appear in the body of sorting algorithms that have been developed over the years. In this way, sorting is also a problem of historical interest.
- We can prove a nontrivial lower bound for sorting (as we shall do in Chapter 8). Our best upper bounds match the lower bound asymptotically, and so we know that our sorting algorithms are asymptotically optimal. Moreover, we can use the lower bound for sorting to prove lower bounds for certain other problems.
- Many engineering issues come to the fore when implementing sorting algorithms. The fastest sorting program for a particular situation may depend on many factors, such as prior knowledge about the keys and satellite data, the memory hierarchy (caches and virtual memory) of the host computer, and the software environment. Many of these issues are best dealt with at the algorithmic level, rather than by "tweaking" the code.

Sorting algorithms

We introduced two algorithms that sort n real numbers in Chapter 2. Insertion sort takes $\Theta(n^2)$ time in the worst case. Because its inner loops are tight, however, it is a fast in-place sorting algorithm for small input sizes. (Recall that a sorting algorithm sorts ***in place*** if only a constant number of elements of the input array are ever stored outside the array.) Merge sort has a better asymptotic running time, $\Theta(n \lg n)$, but the MERGE procedure it uses does not operate in place.

In this part, we shall introduce two more algorithms that sort arbitrary real numbers. Heapsort, presented in Chapter 6, sorts n numbers in place in $O(n \lg n)$ time. It uses an important data structure, called a heap, with which we can also implement a priority queue.

Quicksort, in Chapter 7, also sorts n numbers in place, but its worst-case running time is $\Theta(n^2)$. Its expected running time is $\Theta(n \lg n)$, however, and it generally outperforms heapsort in practice. Like insertion sort, quicksort has tight code, and so the hidden constant factor in its running time is small. It is a popular algorithm for sorting large input arrays.

Insertion sort, merge sort, heapsort, and quicksort are all comparison sorts: they determine the sorted order of an input array by comparing elements. Chapter 8 begins by introducing the decision-tree model in order to study the performance limitations of comparison sorts. Using this model, we prove a lower bound of $\Omega(n \lg n)$ on the worst-case running time of any comparison sort on n inputs, thus showing that heapsort and merge sort are asymptotically optimal comparison sorts.

Chapter 8 then goes on to show that we can beat this lower bound of $\Omega(n \lg n)$ if we can gather information about the sorted order of the input by means other than comparing elements. The counting sort algorithm, for example, assumes that the input numbers are in the set $\{0, 1, \ldots, k\}$. By using array indexing as a tool for determining relative order, counting sort can sort n numbers in $\Theta(k + n)$ time. Thus, when $k = O(n)$, counting sort runs in time that is linear in the size of the input array. A related algorithm, radix sort, can be used to extend the range of counting sort. If there are n integers to sort, each integer has d digits, and each digit can take on up to k possible values, then radix sort can sort the numbers in $\Theta(d(n + k))$ time. When d is a constant and k is $O(n)$, radix sort runs in linear time. A third algorithm, bucket sort, requires knowledge of the probabilistic distribution of numbers in the input array. It can sort n real numbers uniformly distributed in the half-open interval $[0, 1)$ in average-case $O(n)$ time.

The following table summarizes the running times of the sorting algorithms from Chapters 2 and 6–8. As usual, n denotes the number of items to sort. For counting sort, the items to sort are integers in the set $\{0, 1, \ldots, k\}$. For radix sort, each item is a d-digit number, where each digit takes on k possible values. For bucket sort, we assume that the keys are real numbers uniformly distributed in the half-open interval $[0, 1)$. The rightmost column gives the average-case or expected running time, indicating which it gives when it differs from the worst-case running time. We omit the average-case running time of heapsort because we do not analyze it in this book.

Algorithm	Worst-case running time	Average-case/expected running time
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Heapsort	$O(n \lg n)$	—
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (expected)
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (average-case)

Order statistics

The ith order statistic of a set of n numbers is the ith smallest number in the set. We can, of course, select the ith order statistic by sorting the input and indexing the ith element of the output. With no assumptions about the input distribution, this method runs in $\Omega(n \lg n)$ time, as the lower bound proved in Chapter 8 shows.

In Chapter 9, we show that we can find the ith smallest element in $O(n)$ time, even when the elements are arbitrary real numbers. We present a randomized algorithm with tight pseudocode that runs in $\Theta(n^2)$ time in the worst case, but whose expected running time is $O(n)$. We also give a more complicated algorithm that runs in $O(n)$ worst-case time.

Background

Although most of this part does not rely on difficult mathematics, some sections do require mathematical sophistication. In particular, analyses of quicksort, bucket sort, and the order-statistic algorithm use probability, which is reviewed in Appendix C, and the material on probabilistic analysis and randomized algorithms in Chapter 5. The analysis of the worst-case linear-time algorithm for order statistics involves somewhat more sophisticated mathematics than the other worst-case analyses in this part.

8 Sorting in Linear Time

We have now introduced several algorithms that can sort n numbers in $O(n \lg n)$ time. Merge sort and heapsort achieve this upper bound in the worst case; quicksort achieves it on average. Moreover, for each of these algorithms, we can produce a sequence of n input numbers that causes the algorithm to run in $\Omega(n \lg n)$ time.

These algorithms share an interesting property: *the sorted order they determine is based only on comparisons between the input elements*. We call such sorting algorithms ***comparison sorts***. All the sorting algorithms introduced thus far are comparison sorts.

In Section 8.1, we shall prove that any comparison sort must make $\Omega(n \lg n)$ comparisons in the worst case to sort n elements. Thus, merge sort and heapsort are asymptotically optimal, and no comparison sort exists that is faster by more than a constant factor.

Sections 8.2, 8.3, and 8.4 examine three sorting algorithms—counting sort, radix sort, and bucket sort—that run in linear time. Of course, these algorithms use operations other than comparisons to determine the sorted order. Consequently, the $\Omega(n \lg n)$ lower bound does not apply to them.

8.1 Lower bounds for sorting

In a comparison sort, we use only comparisons between elements to gain order information about an input sequence $\langle a_1, a_2, \ldots, a_n \rangle$. That is, given two elements a_i and a_j, we perform one of the tests $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$, or $a_i > a_j$ to determine their relative order. We may not inspect the values of the elements or gain order information about them in any other way.

In this section, we assume without loss of generality that all the input elements are distinct. Given this assumption, comparisons of the form $a_i = a_j$ are useless, so we can assume that no comparisons of this form are made. We also note that the comparisons $a_i \leq a_j$, $a_i \geq a_j$, $a_i > a_j$, and $a_i < a_j$ are all equivalent in that

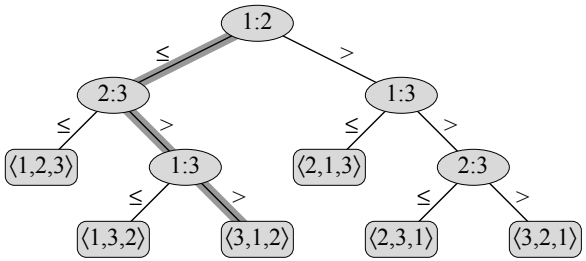


Figure 8.1 The decision tree for insertion sort operating on three elements. An internal node annotated by $i:j$ indicates a comparison between a_i and a_j. A leaf annotated by the permutation $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$ indicates the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$. The shaded path indicates the decisions made when sorting the input sequence $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$; the permutation $\langle 3, 1, 2 \rangle$ at the leaf indicates that the sorted ordering is $a_3 = 5 \leq a_1 = 6 \leq a_2 = 8$. There are $3! = 6$ possible permutations of the input elements, and so the decision tree must have at least 6 leaves.

they yield identical information about the relative order of a_i and a_j. We therefore assume that all comparisons have the form $a_i \leq a_j$.

The decision-tree model

We can view comparison sorts abstractly in terms of decision trees. A ***decision tree*** is a full binary tree that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size. Control, data movement, and all other aspects of the algorithm are ignored. Figure 8.1 shows the decision tree corresponding to the insertion sort algorithm from Section 2.1 operating on an input sequence of three elements.

In a decision tree, we annotate each internal node by $i:j$ for some i and j in the range $1 \leq i, j \leq n$, where n is the number of elements in the input sequence. We also annotate each leaf by a permutation $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$. (See Section C.1 for background on permutations.) The execution of the sorting algorithm corresponds to tracing a simple path from the root of the decision tree down to a leaf. Each internal node indicates a comparison $a_i \leq a_j$. The left subtree then dictates subsequent comparisons once we know that $a_i \leq a_j$, and the right subtree dictates subsequent comparisons knowing that $a_i > a_j$. When we come to a leaf, the sorting algorithm has established the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$. Because any correct sorting algorithm must be able to produce each permutation of its input, each of the $n!$ permutations on n elements must appear as one of the leaves of the decision tree for a comparison sort to be correct. Furthermore, each of these leaves must be reachable from the root by a downward path corresponding to an actual

execution of the comparison sort. (We shall refer to such leaves as "reachable.") Thus, we shall consider only decision trees in which each permutation appears as a reachable leaf.

A lower bound for the worst case

The length of the longest simple path from the root of a decision tree to any of its reachable leaves represents the worst-case number of comparisons that the corresponding sorting algorithm performs. Consequently, the worst-case number of comparisons for a given comparison sort algorithm equals the height of its decision tree. A lower bound on the heights of all decision trees in which each permutation appears as a reachable leaf is therefore a lower bound on the running time of any comparison sort algorithm. The following theorem establishes such a lower bound.

Theorem 8.1
Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

Proof From the preceding discussion, it suffices to determine the height of a decision tree in which each permutation appears as a reachable leaf. Consider a decision tree of height h with l reachable leaves corresponding to a comparison sort on n elements. Because each of the $n!$ permutations of the input appears as some leaf, we have $n! \leq l$. Since a binary tree of height h has no more than 2^h leaves, we have

$$n! \leq l \leq 2^h ,$$

which, by taking logarithms, implies

$$\begin{aligned} h &\geq \lg(n!) && \text{(since the lg function is monotonically increasing)} \\ &= \Omega(n \lg n) && \text{(by equation (3.19))} . \end{aligned}$$

■

Corollary 8.2
Heapsort and merge sort are asymptotically optimal comparison sorts.

Proof The $O(n \lg n)$ upper bounds on the running times for heapsort and merge sort match the $\Omega(n \lg n)$ worst-case lower bound from Theorem 8.1. ■

Exercises

8.1-1
What is the smallest possible depth of a leaf in a decision tree for a comparison sort?

3 Growth of Functions

The order of growth of the running time of an algorithm, defined in Chapter 2, gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms. Once the input size n becomes large enough, merge sort, with its $\Theta(n \lg n)$ worst-case running time, beats insertion sort, whose worst-case running time is $\Theta(n^2)$. Although we can sometimes determine the exact running time of an algorithm, as we did for insertion sort in Chapter 2, the extra precision is not usually worth the effort of computing it. For large enough inputs, the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself.

When we look at input sizes large enough to make only the order of growth of the running time relevant, we are studying the ***asymptotic*** efficiency of algorithms. That is, we are concerned with how the running time of an algorithm increases with the size of the input *in the limit*, as the size of the input increases without bound. Usually, an algorithm that is asymptotically more efficient will be the best choice for all but very small inputs.

This chapter gives several standard methods for simplifying the asymptotic analysis of algorithms. The next section begins by defining several types of "asymptotic notation," of which we have already seen an example in Θ-notation. We then present several notational conventions used throughout this book, and finally we review the behavior of functions that commonly arise in the analysis of algorithms.

3.1 Asymptotic notation

The notations we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of natural numbers $\mathbb{N} = \{0, 1, 2, \ldots\}$. Such notations are convenient for describing the worst-case running-time function $T(n)$, which usually is defined only on integer input sizes. We sometimes find it convenient, however, to *abuse* asymptotic notation in a va-

riety of ways. For example, we might extend the notation to the domain of real numbers or, alternatively, restrict it to a subset of the natural numbers. We should make sure, however, to understand the precise meaning of the notation so that when we abuse, we do not *misuse* it. This section defines the basic asymptotic notations and also introduces some common abuses.

Asymptotic notation, functions, and running times

We will use asymptotic notation primarily to describe the running times of algorithms, as when we wrote that insertion sort's worst-case running time is $\Theta(n^2)$. Asymptotic notation actually applies to functions, however. Recall that we characterized insertion sort's worst-case running time as $an^2 + bn + c$, for some constants a, b, and c. By writing that insertion sort's running time is $\Theta(n^2)$, we abstracted away some details of this function. Because asymptotic notation applies to functions, what we were writing as $\Theta(n^2)$ was the function $an^2 + bn + c$, which in that case happened to characterize the worst-case running time of insertion sort.

In this book, the functions to which we apply asymptotic notation will usually characterize the running times of algorithms. But asymptotic notation can apply to functions that characterize some other aspect of algorithms (the amount of space they use, for example), or even to functions that have nothing whatsoever to do with algorithms.

Even when we use asymptotic notation to apply to the running time of an algorithm, we need to understand *which* running time we mean. Sometimes we are interested in the worst-case running time. Often, however, we wish to characterize the running time no matter what the input. In other words, we often wish to make a blanket statement that covers all inputs, not just the worst case. We shall see asymptotic notations that are well suited to characterizing running times no matter what the input.

Θ-notation

In Chapter 2, we found that the worst-case running time of insertion sort is $T(n) = \Theta(n^2)$. Let us define what this notation means. For a given function $g(n)$, we denote by $\Theta(g(n))$ the *set of functions*

$$\Theta(g(n)) = \{f(n) : \text{ there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\}.$$[1]

[1] Within set notation, a colon means "such that."

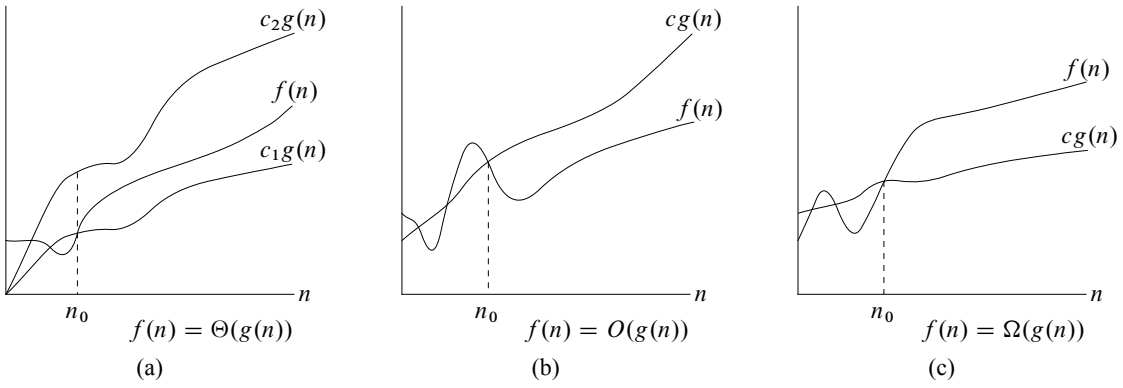


Figure 3.1 Graphic examples of the Θ, O, and Ω notations. In each part, the value of n_0 shown is the minimum possible value; any greater value would also work. **(a)** Θ-notation bounds a function to within constant factors. We write $f(n) = \Theta(g(n))$ if there exist positive constants n_0, c_1, and c_2 such that at and to the right of n_0, the value of $f(n)$ always lies between $c_1 g(n)$ and $c_2 g(n)$ inclusive. **(b)** O-notation gives an upper bound for a function to within a constant factor. We write $f(n) = O(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0, the value of $f(n)$ always lies on or below $cg(n)$. **(c)** Ω-notation gives a lower bound for a function to within a constant factor. We write $f(n) = \Omega(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0, the value of $f(n)$ always lies on or above $cg(n)$.

A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be "sandwiched" between $c_1 g(n)$ and $c_2 g(n)$, for sufficiently large n. Because $\Theta(g(n))$ is a set, we could write "$f(n) \in \Theta(g(n))$" to indicate that $f(n)$ is a member of $\Theta(g(n))$. Instead, we will usually write "$f(n) = \Theta(g(n))$" to express the same notion. You might be confused because we abuse equality in this way, but we shall see later in this section that doing so has its advantages.

Figure 3.1(a) gives an intuitive picture of functions $f(n)$ and $g(n)$, where $f(n) = \Theta(g(n))$. For all values of n at and to the right of n_0, the value of $f(n)$ lies at or above $c_1 g(n)$ and at or below $c_2 g(n)$. In other words, for all $n \geq n_0$, the function $f(n)$ is equal to $g(n)$ to within a constant factor. We say that $g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

The definition of $\Theta(g(n))$ requires that every member $f(n) \in \Theta(g(n))$ be ***asymptotically nonnegative***, that is, that $f(n)$ be nonnegative whenever n is sufficiently large. (An ***asymptotically positive*** function is one that is positive for all sufficiently large n.) Consequently, the function $g(n)$ itself must be asymptotically nonnegative, or else the set $\Theta(g(n))$ is empty. We shall therefore assume that every function used within Θ-notation is asymptotically nonnegative. This assumption holds for the other asymptotic notations defined in this chapter as well.

In Chapter 2, we introduced an informal notion of Θ-notation that amounted to throwing away lower-order terms and ignoring the leading coefficient of the highest-order term. Let us briefly justify this intuition by using the formal definition to show that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$. To do so, we must determine positive constants c_1, c_2, and n_0 such that

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$$

for all $n \geq n_0$. Dividing by n^2 yields

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2 .$$

We can make the right-hand inequality hold for any value of $n \geq 1$ by choosing any constant $c_2 \geq 1/2$. Likewise, we can make the left-hand inequality hold for any value of $n \geq 7$ by choosing any constant $c_1 \leq 1/14$. Thus, by choosing $c_1 = 1/14$, $c_2 = 1/2$, and $n_0 = 7$, we can verify that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$. Certainly, other choices for the constants exist, but the important thing is that *some* choice exists. Note that these constants depend on the function $\frac{1}{2}n^2 - 3n$; a different function belonging to $\Theta(n^2)$ would usually require different constants.

We can also use the formal definition to verify that $6n^3 \neq \Theta(n^2)$. Suppose for the purpose of contradiction that c_2 and n_0 exist such that $6n^3 \leq c_2 n^2$ for all $n \geq n_0$. But then dividing by n^2 yields $n \leq c_2/6$, which cannot possibly hold for arbitrarily large n, since c_2 is constant.

Intuitively, the lower-order terms of an asymptotically positive function can be ignored in determining asymptotically tight bounds because they are insignificant for large n. When n is large, even a tiny fraction of the highest-order term suffices to dominate the lower-order terms. Thus, setting c_1 to a value that is slightly smaller than the coefficient of the highest-order term and setting c_2 to a value that is slightly larger permits the inequalities in the definition of Θ-notation to be satisfied. The coefficient of the highest-order term can likewise be ignored, since it only changes c_1 and c_2 by a constant factor equal to the coefficient.

As an example, consider any quadratic function $f(n) = an^2 + bn + c$, where a, b, and c are constants and $a > 0$. Throwing away the lower-order terms and ignoring the constant yields $f(n) = \Theta(n^2)$. Formally, to show the same thing, we take the constants $c_1 = a/4$, $c_2 = 7a/4$, and $n_0 = 2 \cdot \max(|b|/a, \sqrt{|c|/a})$. You may verify that $0 \leq c_1 n^2 \leq an^2 + bn + c \leq c_2 n^2$ for all $n \geq n_0$. In general, for any polynomial $p(n) = \sum_{i=0}^{d} a_i n^i$, where the a_i are constants and $a_d > 0$, we have $p(n) = \Theta(n^d)$ (see Problem 3-1).

Since any constant is a degree-0 polynomial, we can express any constant function as $\Theta(n^0)$, or $\Theta(1)$. This latter notation is a minor abuse, however, because the

expression does not indicate what variable is tending to infinity.[2] We shall often use the notation $\Theta(1)$ to mean either a constant or a constant function with respect to some variable.

O-notation

The Θ-notation asymptotically bounds a function from above and below. When we have only an ***asymptotic upper bound***, we use O-notation. For a given function $g(n)$, we denote by $O(g(n))$ (pronounced "big-oh of g of n" or sometimes just "oh of g of n") the set of functions

$$O(g(n)) = \{f(n) : \text{ there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\} .$$

We use O-notation to give an upper bound on a function, to within a constant factor. Figure 3.1(b) shows the intuition behind O-notation. For all values n at and to the right of n_0, the value of the function $f(n)$ is on or below $cg(n)$.

We write $f(n) = O(g(n))$ to indicate that a function $f(n)$ is a member of the set $O(g(n))$. Note that $f(n) = \Theta(g(n))$ implies $f(n) = O(g(n))$, since Θ-notation is a stronger notion than O-notation. Written set-theoretically, we have $\Theta(g(n)) \subseteq O(g(n))$. Thus, our proof that any quadratic function $an^2 + bn + c$, where $a > 0$, is in $\Theta(n^2)$ also shows that any such quadratic function is in $O(n^2)$. What may be more surprising is that when $a > 0$, any *linear* function $an + b$ is in $O(n^2)$, which is easily verified by taking $c = a + |b|$ and $n_0 = \max(1, -b/a)$.

If you have seen O-notation before, you might find it strange that we should write, for example, $n = O(n^2)$. In the literature, we sometimes find O-notation informally describing asymptotically tight bounds, that is, what we have defined using Θ-notation. In this book, however, when we write $f(n) = O(g(n))$, we are merely claiming that some constant multiple of $g(n)$ is an asymptotic upper bound on $f(n)$, with no claim about how tight an upper bound it is. Distinguishing asymptotic upper bounds from asymptotically tight bounds is standard in the algorithms literature.

Using O-notation, we can often describe the running time of an algorithm merely by inspecting the algorithm's overall structure. For example, the doubly nested loop structure of the insertion sort algorithm from Chapter 2 immediately yields an $O(n^2)$ upper bound on the worst-case running time: the cost of each iteration of the inner loop is bounded from above by $O(1)$ (constant), the indices i

---

[2] The real problem is that our ordinary notation for functions does not distinguish functions from values. In λ-calculus, the parameters to a function are clearly specified: the function n^2 could be written as $\lambda n.n^2$, or even $\lambda r.r^2$. Adopting a more rigorous notation, however, would complicate algebraic manipulations, and so we choose to tolerate the abuse.

and j are both at most n, and the inner loop is executed at most once for each of the n^2 pairs of values for i and j.

Since O-notation describes an upper bound, when we use it to bound the worst-case running time of an algorithm, we have a bound on the running time of the algorithm on every input—the blanket statement we discussed earlier. Thus, the $O(n^2)$ bound on worst-case running time of insertion sort also applies to its running time on every input. The $\Theta(n^2)$ bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion sort on *every* input. For example, we saw in Chapter 2 that when the input is already sorted, insertion sort runs in $\Theta(n)$ time.

Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n, the actual running time varies, depending on the particular input of size n. When we say "the running time is $O(n^2)$," we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n, no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

Ω-notation

Just as O-notation provides an asymptotic *upper* bound on a function, Ω-notation provides an ***asymptotic lower bound***. For a given function $g(n)$, we denote by $\Omega(g(n))$ (pronounced "big-omega of g of n" or sometimes just "omega of g of n") the set of functions

$$\Omega(g(n)) = \{f(n) : \text{ there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\} .$$

Figure 3.1(c) shows the intuition behind Ω-notation. For all values n at or to the right of n_0, the value of $f(n)$ is on or above $cg(n)$.

From the definitions of the asymptotic notations we have seen thus far, it is easy to prove the following important theorem (see Exercise 3.1-5).

Theorem 3.1

For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. ■

As an example of the application of this theorem, our proof that $an^2 + bn + c = \Theta(n^2)$ for any constants a, b, and c, where $a > 0$, immediately implies that $an^2 + bn + c = \Omega(n^2)$ and $an^2 + bn + c = O(n^2)$. In practice, rather than using Theorem 3.1 to obtain asymptotic upper and lower bounds from asymptotically tight bounds, as we did for this example, we usually use it to prove asymptotically tight bounds from asymptotic upper and lower bounds.

When we say that the *running time* (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen for each value of n*, the running time on that input is at least a constant times $g(n)$, for sufficiently large n. Equivalently, we are giving a lower bound on the best-case running time of an algorithm. For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$.

The running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$, since it falls anywhere between a linear function of n and a quadratic function of n. Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the *worst-case* running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.

Asymptotic notation in equations and inequalities

We have already seen how asymptotic notation can be used within mathematical formulas. For example, in introducing O-notation, we wrote "$n = O(n^2)$." We might also write $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$. How do we interpret such formulas?

When the asymptotic notation stands alone (that is, not within a larger formula) on the right-hand side of an equation (or inequality), as in $n = O(n^2)$, we have already defined the equal sign to mean set membership: $n \in O(n^2)$. In general, however, when asymptotic notation appears in a formula, we interpret it as standing for some anonymous function that we do not care to name. For example, the formula $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ means that $2n^2 + 3n + 1 = 2n^2 + f(n)$, where $f(n)$ is some function in the set $\Theta(n)$. In this case, we let $f(n) = 3n + 1$, which indeed is in $\Theta(n)$.

Using asymptotic notation in this manner can help eliminate inessential detail and clutter in an equation. For example, in Chapter 2 we expressed the worst-case running time of merge sort as the recurrence

$$T(n) = 2T(n/2) + \Theta(n) .$$

If we are interested only in the asymptotic behavior of $T(n)$, there is no point in specifying all the lower-order terms exactly; they are all understood to be included in the anonymous function denoted by the term $\Theta(n)$.

The number of anonymous functions in an expression is understood to be equal to the number of times the asymptotic notation appears. For example, in the expression

$$\sum_{i=1}^{n} O(i) ,$$

there is only a single anonymous function (a function of i). This expression is thus *not* the same as $O(1) + O(2) + \cdots + O(n)$, which doesn't really have a clean interpretation.

In some cases, asymptotic notation appears on the left-hand side of an equation, as in

$$2n^2 + \Theta(n) = \Theta(n^2) .$$

We interpret such equations using the following rule: *No matter how the anonymous functions are chosen on the left of the equal sign, there is a way to choose the anonymous functions on the right of the equal sign to make the equation valid.* Thus, our example means that for *any* function $f(n) \in \Theta(n)$, there is *some* function $g(n) \in \Theta(n^2)$ such that $2n^2 + f(n) = g(n)$ for all n. In other words, the right-hand side of an equation provides a coarser level of detail than the left-hand side.

We can chain together a number of such relationships, as in

$$\begin{aligned} 2n^2 + 3n + 1 &= 2n^2 + \Theta(n) \\ &= \Theta(n^2) . \end{aligned}$$

We can interpret each equation separately by the rules above. The first equation says that there is *some* function $f(n) \in \Theta(n)$ such that $2n^2 + 3n + 1 = 2n^2 + f(n)$ for all n. The second equation says that for *any* function $g(n) \in \Theta(n)$ (such as the $f(n)$ just mentioned), there is *some* function $h(n) \in \Theta(n^2)$ such that $2n^2 + g(n) = h(n)$ for all n. Note that this interpretation implies that $2n^2 + 3n + 1 = \Theta(n^2)$, which is what the chaining of equations intuitively gives us.

o-notation

The asymptotic upper bound provided by O-notation may or may not be asymptotically tight. The bound $2n^2 = O(n^2)$ is asymptotically tight, but the bound $2n = O(n^2)$ is not. We use o-notation to denote an upper bound that is not asymptotically tight. We formally define $o(g(n))$ ("little-oh of g of n") as the set

$$\begin{aligned} o(g(n)) = \{ f(n) : &\text{ for any positive constant } c > 0, \text{ there exists a constant} \\ &n_0 > 0 \text{ such that } 0 \le f(n) < cg(n) \text{ for all } n \ge n_0 \} . \end{aligned}$$

For example, $2n = o(n^2)$, but $2n^2 \neq o(n^2)$.

The definitions of O-notation and o-notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $0 \le f(n) \le cg(n)$ holds for *some* constant $c > 0$, but in $f(n) = o(g(n))$, the bound $0 \le f(n) < cg(n)$ holds for *all* constants $c > 0$. Intuitively, in o-notation, the function $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity; that is,

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \,. \tag{3.1}$$

Some authors use this limit as a definition of the o-notation; the definition in this book also restricts the anonymous functions to be asymptotically nonnegative.

ω-notation

By analogy, ω-notation is to Ω-notation as o-notation is to O-notation. We use ω-notation to denote a lower bound that is not asymptotically tight. One way to define it is by

$f(n) \in \omega(g(n))$ if and only if $g(n) \in o(f(n))$.

Formally, however, we define $\omega(g(n))$ ("little-omega of g of n") as the set

$$\omega(g(n)) = \{f(n) : \text{ for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \le cg(n) < f(n) \text{ for all } n \ge n_0\} \,.$$

For example, $n^2/2 = \omega(n)$, but $n^2/2 \neq \omega(n^2)$. The relation $f(n) = \omega(g(n))$ implies that

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \,,$$

if the limit exists. That is, $f(n)$ becomes arbitrarily large relative to $g(n)$ as n approaches infinity.

Comparing functions

Many of the relational properties of real numbers apply to asymptotic comparisons as well. For the following, assume that $f(n)$ and $g(n)$ are asymptotically positive.

Transitivity:

$$\begin{aligned}
&f(n) = \Theta(g(n)) \text{ and } g(n) = \Theta(h(n)) \quad \text{imply} \quad f(n) = \Theta(h(n)) \,,\\
&f(n) = O(g(n)) \text{ and } g(n) = O(h(n)) \quad \text{imply} \quad f(n) = O(h(n)) \,,\\
&f(n) = \Omega(g(n)) \text{ and } g(n) = \Omega(h(n)) \quad \text{imply} \quad f(n) = \Omega(h(n)) \,,\\
&f(n) = o(g(n)) \text{ and } g(n) = o(h(n)) \quad \text{imply} \quad f(n) = o(h(n)) \,,\\
&f(n) = \omega(g(n)) \text{ and } g(n) = \omega(h(n)) \quad \text{imply} \quad f(n) = \omega(h(n)) \,.
\end{aligned}$$

Reflexivity:

$$\begin{aligned}
f(n) &= \Theta(f(n)) \,,\\
f(n) &= O(f(n)) \,,\\
f(n) &= \Omega(f(n)) \,.
\end{aligned}$$

Symmetry:

$$f(n) = \Theta(g(n)) \text{ if and only if } g(n) = \Theta(f(n)) .$$

Transpose symmetry:

$$f(n) = O(g(n)) \text{ if and only if } g(n) = \Omega(f(n)) ,$$
$$f(n) = o(g(n)) \text{ if and only if } g(n) = \omega(f(n)) .$$

Because these properties hold for asymptotic notations, we can draw an analogy between the asymptotic comparison of two functions f and g and the comparison of two real numbers a and b:

$$f(n) = O(g(n)) \text{ is like } a \leq b ,$$
$$f(n) = \Omega(g(n)) \text{ is like } a \geq b ,$$
$$f(n) = \Theta(g(n)) \text{ is like } a = b ,$$
$$f(n) = o(g(n)) \text{ is like } a < b ,$$
$$f(n) = \omega(g(n)) \text{ is like } a > b .$$

We say that $f(n)$ is ***asymptotically smaller*** than $g(n)$ if $f(n) = o(g(n))$, and $f(n)$ is ***asymptotically larger*** than $g(n)$ if $f(n) = \omega(g(n))$.

One property of real numbers, however, does not carry over to asymptotic notation:

Trichotomy: For any two real numbers a and b, exactly one of the following must hold: $a < b$, $a = b$, or $a > b$.

Although any two real numbers can be compared, not all functions are asymptotically comparable. That is, for two functions $f(n)$ and $g(n)$, it may be the case that neither $f(n) = O(g(n))$ nor $f(n) = \Omega(g(n))$ holds. For example, we cannot compare the functions n and $n^{1+\sin n}$ using asymptotic notation, since the value of the exponent in $n^{1+\sin n}$ oscillates between 0 and 2, taking on all values in between.

Exercises

3.1-1

Let $f(n)$ and $g(n)$ be asymptotically nonnegative functions. Using the basic definition of Θ-notation, prove that $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

3.1-2

Show that for any real constants a and b, where $b > 0$,

$$(n + a)^b = \Theta(n^b) . \tag{3.2}$$

S1.1 Sorting as a precise contract

Input, output, and identity

A sorting algorithm rearranges records according to a key and an ordering rule. For ascending numeric order, every adjacent output pair must satisfy A[i] <= A[i+1]. The output must also contain exactly the original records, including every repeated occurrence. These are independent obligations: a sorted-looking array can still be wrong if it loses or duplicates a record.

The key is the part used for comparison; satellite data are the fields that travel with that key. Sorting a list of (score, student_id) records by score must move each entire record. Sorting only the score column destroys the association with the student. The ordering of equal scores is a separate stability question.

A contract before an implementation

State whether the function modifies the input, returns a fresh sequence, accepts an empty input, and permits repeated keys. Define the comparator and its treatment of special values. A numeric routine cannot silently switch to lexicographic string order: "10" precedes "2" lexicographically, although 2 is smaller numerically.

For a comparator cmp, sortedness means cmp(A[i], A[i+1]) <= 0. Require a consistent ordering. If the comparator gives contradictory results, no sorting algorithm can supply a meaningful ordering guarantee.

Worked interpretation

Input records (3,a), (1,b), (3,c), (2,d) can produce (1,b), (2,d), (3,a), (3,c). This output is sorted, preserves identities, and is stable. Reversing the two records with key 3 remains numerically sorted but loses stability.

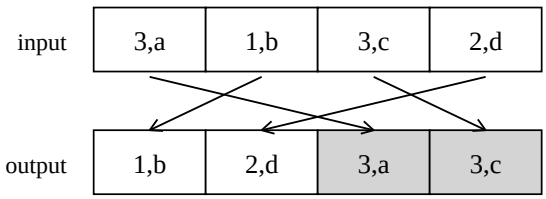


Figure S1.1 Record identity is preserved while the key determines order.

New explanatory material; original source: CLRS 1.1; Part II introduction

S1.2 Stability, tie rules, and multiple keys

What stability guarantees

A stable sort preserves the input order of records whose keys compare equal. Stability concerns record identity, not whether repeated numeric values remain present. With input (2,a), (1,b), (2,c), a stable ascending output places a before c. An unstable implementation may still be completely correct under a contract requiring only sortedness and a permutation.

Local rules with global consequences

Adjacent-swap bubble sort is stable when it swaps only if the left key is strictly greater. Insertion sort is stable when it shifts only keys strictly greater than the saved key. A merge is stable when it takes from the left run on equality. Stable counting sort using cumulative end positions processes input from right to left. These rules are part of an implementation, not consequences of the algorithm name alone.

The usual selection sort is unstable because a long-distance swap can move one record past an equal record. Selecting the first minimum does not repair that effect. A stable selection variant extracts the first minimum and shifts the intervening segment instead of exchanging the endpoints.

Multiple-key sorting

Suppose the required order is department first, then name. One option is a single lexicographic comparator (department, name). Another is to sort stably by name and then stably by department. The second pass preserves the established name order inside each department. Reversing these passes changes the primary key.

If an explicit original-position field is appended to the comparison key, even an unstable implementation can return the same tie order as a stable sort. That changes the comparator and may require extra storage; it does not make the underlying implementation inherently stable.

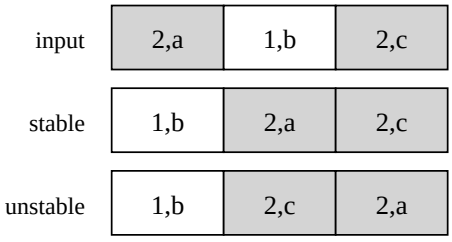


Figure S1.2 Equal keys carry distinct labels; their order is observable.

New explanatory material; original source: CLRS 8.2; original supplementary examples

S1.3 Cost models: time, space, and data movement

Choose the quantities being measured

Let n denote the number of records. Count key comparisons, swaps, shifts, writes, and auxiliary storage separately. A swap of two array entries normally performs two array writes and uses one temporary record; a shift performs one array write. Some conventions count the temporary assignment as a third assignment. State the convention before comparing totals.

The RAM model assigns constant cost to basic operations on machine-sized values. Comparing long strings, moving large records, or evaluating a costly key function can violate a simplistic constant-cost assumption. Sorting indices or pointers may reduce movement of large payloads, but the references still require storage and the output contract changes.

Auxiliary space versus input size

Bubble, ordinary selection, and array insertion sort use a constant number of temporary records, giving O(1) auxiliary space under the standard model. A conventional array merge sort uses O(n) temporary array storage and O(log n) recursion stack depth; its total auxiliary space is O(n). Stable counting sort uses O(n+R) extra storage when R is the number of possible integer keys.

Space complexity is peak simultaneously live storage. Allocating a new buffer at each recursive call does not automatically imply O(n log n) peak memory; allocation lifetimes matter. Repeated allocation can still increase total allocation traffic and practical overhead.

Tradeoffs instead of one universal winner

Selection sort can reduce writes while still performing quadratic comparisons. Insertion sort can exploit nearly sorted input. Merge sort gives a predictable worst-case bound but needs a buffer in its standard array form. Counting sort depends on a suitably small integer range. A choice must combine the input distribution, storage budget, tie-order requirement, and operation costs.



Count these separately; their prices need not be equal.

Figure S1.3 Distinct operation counts describe different resource costs.

New explanatory material; original source: CLRS 2.2; Part II introduction

S1.4 Bounds, input cases, and average-case assumptions

Separate the function from its bound

Big-O is an asymptotic upper bound, Omega a lower bound, and Theta a tight bound. These notations do not themselves mean worst, best, or average case. First define the cost function, such as worst-case comparisons for inputs of length n, and then state a bound for that function.

The fixed-loop bubble sort from CLRS performs n(n-1)/2 key comparisons regardless of initial order. Its best and worst running times are Theta(n^2), even though a sorted input needs no swaps. Adding an early-exit flag changes the best running time to Theta(n), not its quadratic worst case. An answer that simply says "bubble sort is linear in the best case" is incomplete without naming the variant.

Average is a model, not a typical-looking picture

For uniformly random permutations of n distinct keys, each pair is inverted with probability 1/2. The expected number of inversions is therefore n(n-1)/4. Ordinary insertion sort has expected quadratic time under this distribution, and adjacent-swap bubble sort has an expected quadratic number of swaps.

Real inputs may have duplicates, long sorted runs, or a single displaced value. They need not follow the uniform-permutation model. A claimed average bound should identify the distribution, and a measured benchmark should identify the data generation process.

A concrete comparison

For n=8, fixed-loop bubble and standard selection each make 28 key comparisons. Reverse-distinct input needs 28 adjacent swaps in bubble sort, while selection makes at most 7 nontrivial swaps. Equal asymptotic comparison bounds therefore do not imply equal movement costs.

n	4	8	16	32
n(n-1)/2	6	28	120	496
n log2(n)	8	24	64	160

Figure S1.4 Exact comparison counts can differ from qualitative growth labels.

New explanatory material; original source: CLRS 2.2; 3.1

S1.5 Correctness: invariants, termination, and permutations

A complete proof has more than sortedness

To prove a sorting algorithm correct, establish that it terminates, preserves the input multiset or record identities, and produces an ordered output. An array filled entirely with the smallest value is sorted but usually violates preservation. A loop that would eventually establish sortedness is not useful if its stopping rule can fail to terminate.

An invariant is a statement true at a designated point in each iteration. State that point precisely. Prove initialization before the first iteration, maintenance from one iteration to the next, and the useful consequence at termination. A decreasing nonnegative quantity or a finite loop range supplies a separate termination argument.

Three different regions

Selection sort maintains a sorted prefix containing the smallest records. Insertion sort maintains a sorted prefix containing exactly the records originally processed, which need not be globally smallest. A right-to-left bubble pass establishes the next smallest record at the prefix boundary. Confusing these statements leads to a proof that does not match the code.

For merging, the output prefix consists of the smallest consumed records from two already sorted runs, and the next output record must be the smaller of the two unconsumed heads. For counting sort, counts and cumulative positions reserve disjoint output intervals for each key.

Useful counterexamples

If an alleged invariant says the first insertion-sort prefix contains the global minimum, test input [5,2,1] immediately after processing [5,2]. The prefix [2,5] is sorted but excludes 1. A small counterexample often reveals an incorrect statement more clearly than an informal explanation.

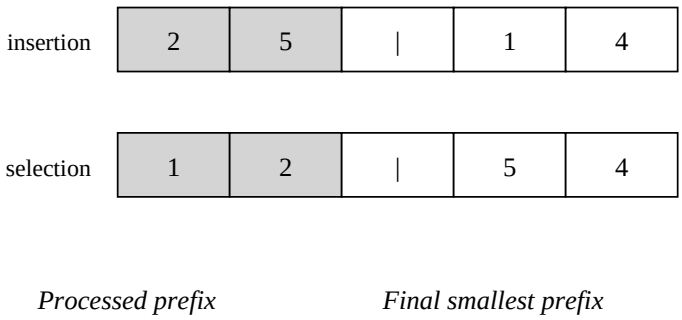


Figure S1.5 A sorted processed prefix differs from a globally final prefix.

New explanatory material; original source: CLRS 2.1; Problems 2-2 and 2-4

S1.6 Input contracts and exceptional cases

Empty, singleton, repeated, and signed keys

An empty sequence and a singleton are already sorted. Return them without invalid indexing. Repeated keys must preserve multiplicities; all-equal input is useful for checking stability and early termination. Comparison sorts naturally handle negative numbers when the comparator is valid. Counting sort needs an explicit integer range and an offset for negative keys.

For a numeric comparator, special floating-point values require a deliberate policy. NaN does not behave like an ordinary ordered number under usual numeric comparisons. Decide whether such values are rejected, placed at an end, or handled by a defined total ordering. Do not assume that an implementation designed for ordinary integers automatically establishes a valid floating-point ordering.

Ascending, descending, and custom keys

For descending bubble sort, swap when the left key is smaller. For descending insertion sort, shift keys smaller than the saved key. For descending selection sort, select a maximum. For stable descending merge, choose the left run when keys are equal. Each adaptation changes the strict direction while preserving the tie rule.

Counting sort can emit buckets in descending key order, but reversing the entire stable ascending output reverses equal-key record order too. To preserve stability, reverse the order of key groups while keeping each group's internal record order.

Preconditions and output behavior

Specify zero-based or one-based indexing, whether ranges are inclusive or half-open, and whether records are copied or rearranged in place. Avoid mixing these conventions inside one procedure. If key extraction is expensive, caching keys is a separate memory-time tradeoff.

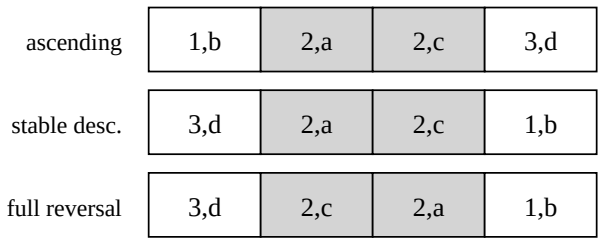


Figure S1.6 Stable descending order reverses key groups, not equal-key identities.

New explanatory material; original source: CLRS 2.1 and 8.2; original supplementary guidance

S1.7 Semantic retrieval: concepts, conditions, and answer structure

From wording to a mathematical concept

The questions "keep equal items in their original order", "preserve ties", and "maintain arrival order for repeated scores" ask about stability. "Use little extra memory", "sort in the same array", and "avoid a buffer" concern auxiliary space. "Almost ordered", "few inversions", and "small disorder" concern adaptivity, although they are not identical input descriptions.

Identify the concept and the implementation variant before retrieving a conclusion. A useful answer records: algorithm, variant, input domain, indexing convention, comparator, cost measure, bound, and stability rule. Missing conditions can convert a correct statement into a misleading answer.

Answer template

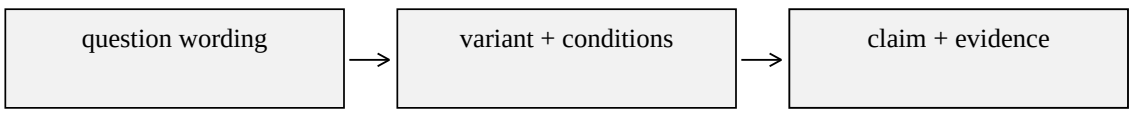
Begin with the direct result. State the condition that makes it true. Explain the mechanism using the relevant invariant or loop. Supply a small example or counterexample. Distinguish the guarantee from a measured performance claim. Point to a named section or figure for evidence instead of guessing a universal ranking.

For "Why is bubble sort O(n) on sorted data?", first identify the early-exit variant. It performs one pass with n-1 comparisons, observes no swaps, and stops. The fixed-loop CLRS variant keeps all passes and is quadratic even on sorted data.

For "Does selection sort stop after finding the smallest item?", distinguish an individual scan from the full algorithm. Each pass finds one minimum; the algorithm still repeats on the remaining suffix. It ordinarily cannot know the minimum before inspecting every unexamined suffix key.

Retrieval boundaries

Keep a definition, its preconditions, and the example that demonstrates it in the same knowledge chunk where possible. For a long proof, connect chunks through explicit algorithm and section names. A citation to an exercise statement does not itself establish a worked solution; use the supplemental proof where supplied.



"already sorted" -> early-exit bubble -> one no-swap pass

Figure S1.7 Retrieve a conditioned claim rather than an isolated complexity label.

New explanatory material; original source: Original supplementary retrieval guidance

S1.8 Choosing and validating a sorting method

A decision based on the contract

Start with the data domain. Dense, bounded integer keys may justify counting sort, provided the range array and output buffer fit memory. Arbitrary comparable records require a comparison method. If a stable quadratic method is acceptable for small or nearly ordered input, insertion sort is a useful candidate. Standard array merge sort supplies stable Theta(n log n) worst-case time when a linear buffer is available.

Selection sort may be attractive in a constrained setting where writes are much more expensive than comparisons and instability is acceptable. Bubble sort is particularly useful for teaching local exchanges and invariants; its optimized variant recognizes sorted input, but it does not provide a generally superior guarantee over insertion sort.

A compact comparison

Fixed-loop bubble: Theta(n^2) comparisons, O(1) space, stable under strict swaps. Early-exit bubble: Theta(n) best time, Theta(n^2) worst time. Selection: Theta(n^2) comparisons in all cases, at most n-1 nontrivial swaps, usually unstable. Insertion: Theta(n+I) time for I inversions, stable under strict shifts. Standard merge: Theta(n log n) time, O(n) space, stable with left-first ties. Counting: Theta(n+R) time, O(n+R) space in the stable output-array form.

Evidence from an implementation

Validate sortedness, identity preservation, and stability separately. Include empty, singleton, all-equal, reversed, and nearly sorted inputs. Instrument comparisons and writes if the question concerns costs. If timing is needed, use the same input copies and disclose input generation, key types, and copy/allocation costs.

Scope and provenance

The retained pages are excerpts from Introduction to Algorithms, third edition. Pages marked Supplement are newly written teaching material, not pages authored by Cormen and coauthors. Source page numbers remain unchanged; supplemental headers show positions in this combined PDF. Adjoining unrelated text remains only where preserving an original whole page requires it.

External corroboration: NIST's Dictionary of Algorithms and Data Structures, "bubble sort" (xlinux.nist.gov/dads/HTML/bubblesort.html). The core definitions, bounds, and original exercises are cited by CLRS section throughout the supplement.

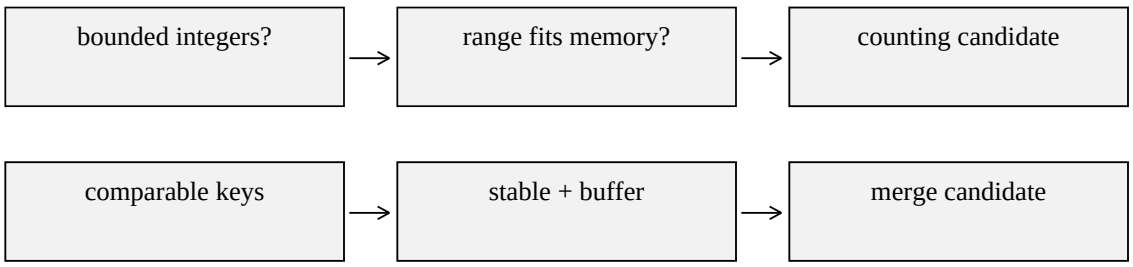


Figure S1.8 Domain, stability, memory, and cost constraints guide selection.

New explanatory material; original source: CLRS 2.2, 8.1, 8.2; original supplementary synthesis

subproblems become sufficiently small. Consider a modification to merge sort in which n/k sublists of length k are sorted using insertion sort and then merged using the standard merging mechanism, where k is a value to be determined.

a. Show that insertion sort can sort the n/k sublists, each of length k, in $\Theta(nk)$ worst-case time.

b. Show how to merge the sublists in $\Theta(n \lg(n/k))$ worst-case time.

c. Given that the modified algorithm runs in $\Theta(nk + n \lg(n/k))$ worst-case time, what is the largest value of k as a function of n for which the modified algorithm has the same running time as standard merge sort, in terms of Θ-notation?

d. How should we choose k in practice?

2-2 Correctness of bubblesort

Bubblesort is a popular, but inefficient, sorting algorithm. It works by repeatedly swapping adjacent elements that are out of order.

```
BUBBLESORT(A)
1  for i = 1 to A.length − 1
2      for j = A.length downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

a. Let A' denote the output of BUBBLESORT(A). To prove that BUBBLESORT is correct, we need to prove that it terminates and that

$$A'[1] \le A'[2] \le \cdots \le A'[n] \,, \tag{2.3}$$

where $n = A.length$. In order to show that BUBBLESORT actually sorts, what else do we need to prove?

The next two parts will prove inequality (2.3).

b. State precisely a loop invariant for the **for** loop in lines 2–4, and prove that this loop invariant holds. Your proof should use the structure of the loop invariant proof presented in this chapter.

c. Using the termination condition of the loop invariant proved in part (b), state a loop invariant for the **for** loop in lines 1–4 that will allow you to prove inequality (2.3). Your proof should use the structure of the loop invariant proof presented in this chapter.

d. What is the worst-case running time of bubblesort? How does it compare to the running time of insertion sort?

2-3 Correctness of Horner's rule

The following code fragment implements Horner's rule for evaluating a polynomial

$$\begin{aligned} P(x) &= \sum_{k=0}^{n} a_k x^k \\ &= a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x a_n)\cdots)) \,, \end{aligned}$$

given the coefficients $a_0, a_1, \ldots, a_n$ and a value for x:

```
1  y = 0
2  for i = n downto 0
3      y = a_i + x · y
```

a. In terms of Θ-notation, what is the running time of this code fragment for Horner's rule?

b. Write pseudocode to implement the naive polynomial-evaluation algorithm that computes each term of the polynomial from scratch. What is the running time of this algorithm? How does it compare to Horner's rule?

c. Consider the following loop invariant:

> At the start of each iteration of the **for** loop of lines 2–3,
>
> $$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k \,.$$
>
> Interpret a summation with no terms as equaling 0. Following the structure of the loop invariant proof presented in this chapter, use this loop invariant to show that, at termination, $y = \sum_{k=0}^{n} a_k x^k$.

d. Conclude by arguing that the given code fragment correctly evaluates a polynomial characterized by the coefficients $a_0, a_1, \ldots, a_n$.

2-4 Inversions

Let $A[1..n]$ be an array of n distinct numbers. If $i < j$ and $A[i] > A[j]$, then the pair (i, j) is called an ***inversion*** of A.

a. List the five inversions of the array $\langle 2, 3, 8, 6, 1 \rangle$.

S2.1 Bubble sort: local exchanges and pass direction

The basic operation

Bubble sort repeatedly examines adjacent records and exchanges an out-of-order pair. For ascending order, swap only when the left key is greater than the right key. Each exchange removes exactly one inversion because the two records are adjacent and only their relative order changes. A nonadjacent exchange does not have that same one-inversion property.

The CLRS direction

Problem 2-2 scans from right to left in each inner loop. The smallest key in the remaining suffix moves left through neighboring positions. After the pass for outer index i, A[i] contains a smallest remaining key, and the finalized prefix grows by one.

Many presentations scan left to right instead. That version moves a largest remaining key toward the right end and grows a finalized suffix. Both are bubble sort, but their intermediate arrays and invariants differ. A trace question must specify the scan direction and what counts as one pass.

A right-to-left pass

For [5,1,4,2,8], start at the pair (2,8), which needs no swap. Then (4,2) swaps, giving [5,1,2,4,8]. The pair (1,2) needs no swap. Finally (5,1) swaps, giving [1,5,2,4,8]. The smallest key 1 is now final at the left boundary. The suffix is not yet sorted.

The first pass need not leave the entire largest-to-smallest arrangement otherwise unchanged. Several records may move, and some incidental positions may happen to be final. The invariant guarantees one boundary record, not that exactly one record changes position.

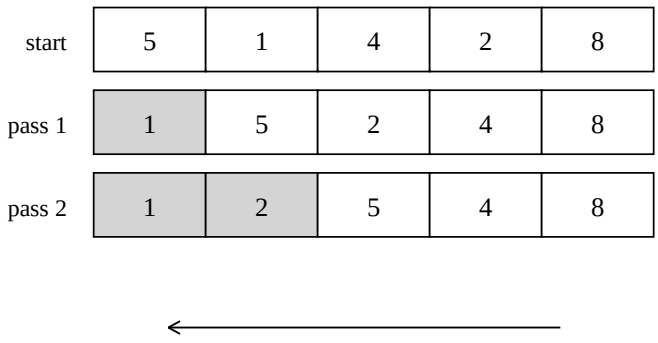


Figure S2.1 Right-to-left passes finalize the smallest remaining key.

New explanatory material; original source: CLRS Problem 2-2; original supplementary explanation

S2.2 Bubble sort: baseline and early-exit pseudocode

Zero-based fixed-loop version

The following procedure corresponds to the scan direction of CLRS, translated into zero-based indexing. Each inner comparison checks the pair ending at j. The boundary j>i keeps j-1 valid.

```
BUBBLE-RTL(A)
  n = length(A)
  for i = 0 to n-2
    for j = n-1 downto i+1
      if key(A[j]) < key(A[j-1])
        exchange A[j] and A[j-1]
```

The outer loop performs n-1 passes for n>=2. This baseline does not check whether the input is already sorted. Empty and singleton inputs execute no pass when the loop conventions treat an empty range as empty.

An optimized left-to-right version

This alternative uses a shrinking right boundary and a flag that must be reset at the beginning of every pass. Strict comparison preserves the order of equal keys.

```
BUBBLE-EARLY(A)
  for end = length(A)-1 downto 1
    swapped = false
    for j = 0 to end-1
      if key(A[j]) > key(A[j+1])
        exchange A[j] and A[j+1]
        swapped = true
    if not swapped
      return
```

Interpret the condition carefully

No swaps during an entire pass mean all adjacent pairs in the active region are ordered. The already finalized suffix is ordered and its keys are no smaller than those still active, so the complete array is sorted. One comparison that needs no swap proves only that one pair is ordered.

S2.3 Bubble sort: a complete trace and operation counts

Follow the exact left-to-right early-exit procedure

Start with [5,1,4,2,8]. In pass 1, exchange (5,1), then (5,4), then (5,2), and leave (5,8) unchanged. The array is [1,4,2,5,8]. Four comparisons and three swaps occurred. The right boundary now excludes 8.

In pass 2, compare (1,4), exchange (4,2), and compare (4,5). The array becomes [1,2,4,5,8]. This pass uses three comparisons and one swap. Because a swap occurred, the algorithm cannot stop using the flag alone, even though the full array is now ordered.

In pass 3, compare (1,2) and (2,4). Neither pair swaps, so the algorithm stops. This pass uses two comparisons and zero swaps. The complete execution uses nine key comparisons and four swaps. A fixed-loop left-to-right version also performs the final one-comparison pass, giving ten comparisons.

Explain a state rather than guess it

At a pass boundary, distinguish the active region from the finalized suffix. During a pass, the largest value encountered so far occupies the right endpoint of the pair just processed. That local property explains why the pass maximum reaches the active right boundary.

The input has four inversions: (5,1), (5,4), (5,2), and (4,2). Exactly four strict adjacent exchanges occur. This supplies an independent check on the trace's swap count, but not on its comparison count; comparisons include pairs already in order.

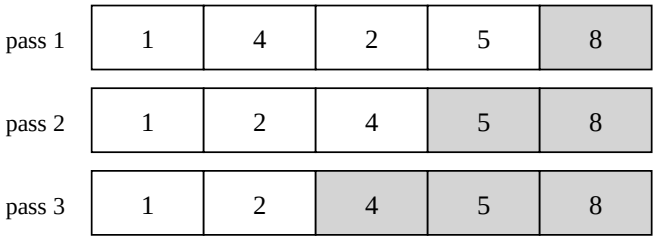


Figure S2.3 The no-swap certification pass is part of the operation count.

New explanatory material; original source: CLRS Problem 2-2; original worked example

S2.4 Bubble sort: correctness of the CLRS version

Inner-loop invariant

Fix outer index i in the zero-based right-to-left version. Immediately before comparing A[j-1] and A[j], A[j] is a smallest key among the records currently in positions j through n-1. Initially j=n-1, so the statement holds for a singleton suffix.

After comparing and possibly exchanging the pair, A[j-1] is a smallest key of positions j-1 through n-1. If the left key is larger, the existing suffix minimum moves left; otherwise the left key is itself no larger than every suffix key. Decrementing j establishes the invariant at the next comparison.

When the inner loop ends, A[i] is a smallest key in the active suffix A[i..n-1]. Exchanges preserve all records. Equal-key records may supply any minimum identity for this invariant; the strict adjacent-swap rule gives stability separately.

Outer-loop invariant

Before pass i, A[0..i-1] is sorted and contains the i smallest records by key, with every prefix key no larger than every remaining suffix key. For i=0, the empty prefix makes the statement trivially true. The inner pass puts a smallest remaining key at A[i], extending the prefix and preserving the relation to the suffix.

After n-1 passes, the prefix contains the n-1 smallest records in order. The single remaining record cannot be smaller than the prefix boundary, so the entire array is ordered. Both loops have finite decreasing or increasing integer ranges, establishing termination. Each exchange is a permutation, establishing preservation.

The additional obligation in Problem 2-2(a)

Showing A'[0] <= A'[1] <= ... <= A'[n-1] is insufficient on its own. Also show that A' is a permutation of A, including repeated record identities. This excludes outputs that fabricate values or erase occurrences.



Figure S2.4 The suffix minimum moves one position left at each comparison.

New explanatory material; original source: CLRS Problem 2-2(a)-(c); original worked proof

S2.5 Bubble sort: best, worst, and expected costs

Fixed comparisons

In the fixed-loop version the pass lengths are n-1, n-2, ..., 1. Their sum is n(n-1)/2. Consequently key comparisons and running time are Theta(n^2) in both the best and worst cases under the constant-cost RAM model. This statement remains true for already sorted and all-equal inputs, despite zero swaps.

Exchanges and inversions

An inversion is a pair of positions p<q with key(A[p])>key(A[q]). Every strict adjacent exchange removes exactly one inversion, and the sorted final array has none. Thus the total number of exchanges is exactly the input inversion count I. With distinct reversed keys, I=n(n-1)/2. With already ordered keys, I=0.

For a uniformly random permutation of distinct keys, each of the n(n-1)/2 pairs is inverted with probability 1/2. Linearity of expectation gives E[I]=n(n-1)/4. An adjacent-swap implementation therefore has expected quadratic exchange cost under this model.

Early termination changes the best case

The early-exit version checks n-1 adjacent pairs and then stops on sorted input, giving Theta(n) time for n>=2. Its worst case still uses quadratic comparisons and exchanges. Its average time on uniformly random distinct permutations is also quadratic, since the expected swap count alone is quadratic and the comparison count is bounded above by a quadratic sum.

One badly placed item can make a left-to-right version expensive even if the rest is ordered. In [2,3,...,n,1], the smallest value moves left only one position per pass. There are n-1 inversions, yet that direction still uses Theta(n^2) comparisons. Therefore a linear inversion count does not imply linear bubble-sort time.

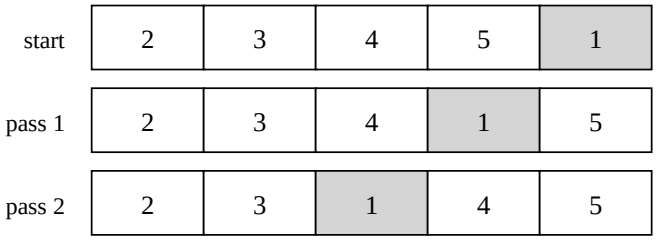


Figure S2.5 A small key moves left slowly under left-to-right scanning.

New explanatory material; original source: CLRS Problem 2-2(d); original derivation

S2.6 Bubble sort: stability and record-safe exchanges

Why strict adjacent exchange is stable

Consider two equal-key records x and y, with x originally before y. Their relative order can change only if they cross. In an adjacent-swap algorithm, crossing would require exchanging x and y when they become neighbors. A condition that swaps only strictly out-of-order keys never exchanges equal keys, so the order of x and y is preserved.

This proof depends on adjacency. Ordinary selection sort can reverse equal-key records by exchanging a third record across them, even when it never directly swaps equal keys. A local no-equality-swap condition alone does not establish stability for arbitrary exchanges.

An unstable modification

For input (2,a), (2,b), a left-to-right condition using >= rather than > exchanges the two records even though their keys are already ordered. This violates stability. In an unbounded repeat-until-no-swaps version, swapping equal keys can also prevent the no-swap termination condition from ever occurring on an all-equal array.

Move the whole record

If records contain key, id, and payload, exchanging only key fields produces corrupted associations. A swap should rearrange entire records or references to records. If key evaluation can change during sorting, the comparator contract is no longer fixed; materialize a consistent key before sorting when appropriate.

For descending stable order, exchange adjacent records only when the left key is strictly smaller. Equal-key record identities retain their original order. Reversing a stable ascending result is not equivalent for records with repeated keys.

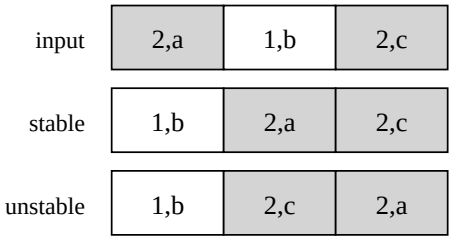


Figure S2.6 Strict comparison preserves labels attached to equal keys.

New explanatory material; original source: CLRS Problem 2-2; original supplementary examples

S2.7 Bubble sort: boundaries, variants, and common failures

Safe boundaries

In a zero-based left-to-right pass ending at end, iterate j from 0 through end-1, and compare A[j] with A[j+1]. Using j=end reads beyond the active boundary. In the CLRS right-to-left version, iterate j from n-1 down through i+1 and compare A[j-1] with A[j]. A loop that reaches j=0 would make j-1 invalid.

After a left-to-right pass, the maximum in the active region is final at end, so decrement end. Do not shrink the left boundary in that same version without an additional proof. The opposite scan direction finalizes a prefix instead.

Useful adaptations

A last-swap boundary stores the right index of the last exchange during a left-to-right pass. All pairs after that location were left ordered and the carried pass maximum makes the following suffix final; the next active end can be that last exchange position. No exchanges mean termination. This can reduce scanning without changing the quadratic worst case.

Cocktail or bidirectional bubble sort alternates left-to-right and right-to-left sweeps and maintains boundaries at both ends. It helps transport small values toward the left as well as large values toward the right. It remains a quadratic method in the worst case and is a different trace from the single-direction listing.

Debugging by smallest counterexample

Test [] and [7] for empty-loop handling, [2,1] for actual swapping, [1,2] for a no-swap pass, and labeled equal records for stability. If an implementation returns after the first comparison needing no swap, [1,3,2] reveals the error: the first pair is ordered but the second is not.

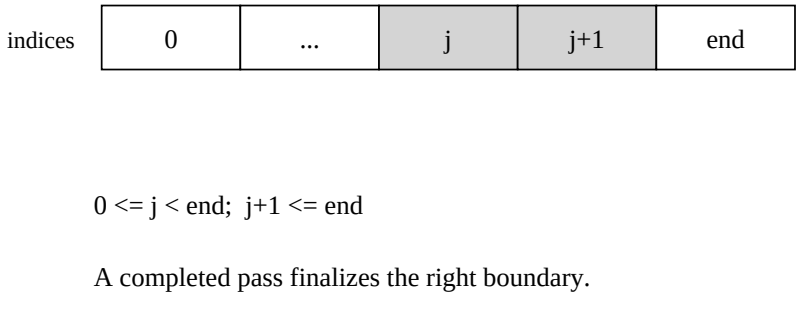


Figure S2.7 The compared pair stays inside the active interval.

New explanatory material; original source: CLRS Problem 2-2; original supplementary implementation guidance

S2.8 Bubble sort: reference implementation and checks

An in-place stable implementation

This function sorts a mutable list in place and returns it for convenience. A key function must be consistent and produce mutually comparable values. The strict comparison implements stable ascending order.

```
def bubble_sort(a, key=lambda x: x):
    for end in range(len(a)-1, 0, -1):
        swapped = False
        for j in range(end):
            if key(a[j]) > key(a[j+1]):
                a[j], a[j+1] = a[j+1], a[j]
                swapped = True
        if not swapped:
            break
    return a
```

Independent properties to check

Sortedness: every adjacent output key is nondecreasing. Preservation: each original record identity occurs exactly once in the output. Stability: for every key, the sequence of identities in the output equals the corresponding sequence in the input. Do not use a set comparison as a multiplicity check; a set discards repeated occurrences.

For [5,1,4,2,8], the output is [1,2,4,5,8]. For labeled input [(2,'a'),(1,'b'),(2,'c')], using the first tuple field as key yields [(1,'b'),(2,'a'),(2,'c')]. Empty and singleton lists return without a comparison. The function does not validate NaN or a malformed comparator; those are contract questions.

Instrument without changing the algorithm

Increment a comparison counter immediately before the if condition and a swap counter only inside the taken branch. For the numeric trace above, the counters are 9 and 4. If key evaluations are also counted, this listing evaluates the key twice per comparison; expensive keys may justify a separate caching design.

S2.9 Bubble sort: deep questions and counterexamples

How can a local algorithm establish global order?

Each completed pass places an extremal remaining record at a final boundary. The outer invariant grows that final region until no unresolved region remains. Alternatively, a complete no-swap pass directly establishes that every active adjacent pair is ordered. Transitivity of the ordering then establishes global order within the region.

Why is it inferior to insertion sort on a displaced minimum?

For left-to-right early-exit bubble sort, [2,3,...,n,1] requires the minimum to travel left one step per pass, yielding quadratic comparisons. Insertion sort saves 1 once and shifts the preceding n-1 records in one insertion, giving linear work for this input. Both perform work related to n-1 inversions, but their scans impose different overhead.

Can the number of passes identify the algorithm?

Not by itself. Selection and bubble can each finalize one boundary record per pass. Inspect the mechanism: bubble exchanges adjacent inverted pairs throughout a scan; selection scans for an extremum and usually performs one long-distance swap. The pass direction and early-exit rule further determine trace behavior.

Does swapping remove exactly one inversion every time?

Only a strict adjacent exchange has that guarantee. If equal adjacent records are exchanged, no strict inversion is removed. A distant selection-sort exchange may change many pair orders at once. The exact inversion argument must name both the comparison and adjacency conditions.

Can bubble sort be useful beyond teaching?

It can be implemented with constant auxiliary storage and a simple local swap rule. A constrained use should justify why those properties matter and why the quadratic worst case is acceptable. Its simplicity is not evidence that it is the fastest general-purpose sorter.

Answer variants

"Will it stop immediately when sorted?" The early-exit listing needs a full no-swap certification pass; fixed-loop CLRS does not stop early. "Does it work on duplicates?" Yes, preserve occurrences and swap strictly for stability. "What if only one element is misplaced?" Its travel direction and displacement matter; that phrase alone does not guarantee few comparisons.

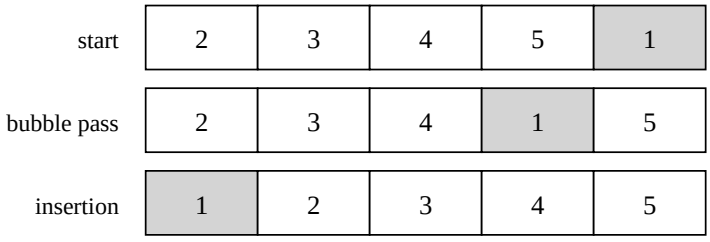


Figure S2.9 A saved key can cross a whole prefix in one insertion.

New explanatory material; original source: CLRS Problem 2-2; original answer bank

order of growth. But for large enough inputs, a $\Theta(n^2)$ algorithm, for example, will run more quickly in the worst case than a $\Theta(n^3)$ algorithm.

Exercises

2.2-1
Express the function $n^3/1000 - 100n^2 - 100n + 3$ in terms of Θ-notation.

2.2-2
Consider sorting n numbers stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A, and exchange it with $A[2]$. Continue in this manner for the first $n-1$ elements of A. Write pseudocode for this algorithm, which is known as ***selection sort***. What loop invariant does this algorithm maintain? Why does it need to run for only the first $n-1$ elements, rather than for all n elements? Give the best-case and worst-case running times of selection sort in Θ-notation.

2.2-3
Consider linear search again (see Exercise 2.1-3). How many elements of the input sequence need to be checked on the average, assuming that the element being searched for is equally likely to be any element in the array? How about in the worst case? What are the average-case and worst-case running times of linear search in Θ-notation? Justify your answers.

2.2-4
How can we modify almost any algorithm to have a good best-case running time?

2.3 Designing algorithms

We can choose from a wide range of algorithm design techniques. For insertion sort, we used an ***incremental*** approach: having sorted the subarray $A[1 \mathinner{..} j-1]$, we inserted the single element $A[j]$ into its proper place, yielding the sorted subarray $A[1 \mathinner{..} j]$.

In this section, we examine an alternative design approach, known as "divide-and-conquer," which we shall explore in more detail in Chapter 4. We'll use divide-and-conquer to design a sorting algorithm whose worst-case running time is much less than that of insertion sort. One advantage of divide-and-conquer algorithms is that their running times are often easily determined using techniques that we will see in Chapter 4.

S3.1 Selection sort: finding the next extremum

Selection rather than adjacent exchange

Selection sort divides the array into a finalized sorted prefix and an unresolved suffix. On pass i, inspect every record in the suffix and remember the position of a smallest key. Exchange that record with A[i]. Continue until only one unresolved record remains.

The scan determines where the minimum is; it does not need to rearrange the array while searching. This is the essential distinction from bubble sort, which performs local exchanges during its scan. Both can move a minimum to the left boundary, but selection uses one direct exchange rather than a chain of adjacent exchanges.

Zero-based pseudocode

```
SELECTION-SORT(A)
  for i = 0 to length(A)-2
    m = i
    for j = i+1 to length(A)-1
      if key(A[j]) < key(A[m])
        m = j
    if m != i
      exchange A[i] and A[m]
```

Why m is an index

Remembering only the minimum key is insufficient when a complete record must be moved. The index m identifies the record and its satellite fields. Reset m=i at each pass; otherwise a minimum position from an earlier finalized region may be reused incorrectly.

Strict < chooses the first encountered minimum among equal candidates. Nevertheless, the distant exchange can disrupt the order of equal records elsewhere. Tie selection and algorithmic stability are different issues.

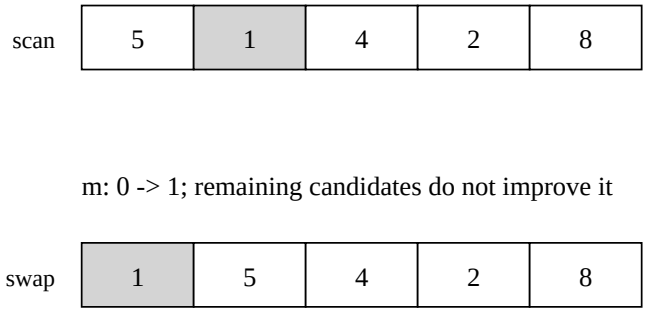


Figure S3.1 A scan remembers a position, then one exchange finalizes the minimum.

New explanatory material; original source: CLRS Exercise 2.2-2; original supplementary explanation

S3.2 Selection sort: a complete trace

Array states at the end of each pass

For [5,1,4,2,8], the first scan examines positions 1 through 4 and identifies 1 at position 1 as the minimum. Exchange it with position 0 to obtain [1,5,4,2,8]. Four key comparisons and one swap occur.

Pass 2 scans [5,4,2,8], finds 2 at position 3, and exchanges it with position 1. The array becomes [1,2,4,5,8]. This uses three comparisons and one swap. The unresolved suffix happens to be sorted, but the standard algorithm does not know that solely from the minimum search.

Pass 3 compares 4 with 5 and 8. The current boundary record 4 is already the minimum, so no nontrivial swap occurs. Pass 4 compares 5 with 8 and also skips the exchange. Total comparisons are 4+3+2+1=10; total nontrivial swaps are 2.

Count the declared operation

A version that always executes exchange(A[i], A[m]) performs four swap calls in this example, including two self-exchanges. The listing with the m!=i guard performs two actual exchanges. Their asymptotic comparison bounds are the same, but their write counts may differ.

The first two passes already produce the final ordered array. A standard selection sort still executes the remaining scans. A routine with an additional sortedness test is a modified algorithm; the test has its own cost and needs a proof of a safe early exit.



Figure S3.2 Two swaps suffice for this example, but all ten comparisons remain.

New explanatory material; original source: CLRS Exercise 2.2-2; original worked example

S3.3 Selection sort: proof of correctness

Inner search invariant

Immediately before processing candidate index j, m identifies a smallest-key record among positions i through j-1. Initially m=i and the scanned interval contains one record. Comparing A[j] with A[m] and updating m only for a smaller key extends the minimum property to positions i through j.

At the end of the scan, m identifies a minimum in the whole active suffix. The scan only reads records and changes an index, so it preserves their positions and identities.

Outer invariant

Before pass i, the prefix A[0..i-1] is ordered, contains i smallest-key records, and every prefix key is no greater than every suffix key. The prefix is empty for i=0, so initialization holds. Exchanging A[i] with the suffix minimum places a smallest remaining key after the existing prefix. The enlarged prefix remains ordered and no larger than the new suffix.

The displaced A[i] remains in the suffix; an exchange cannot lose it. A self-swap can be skipped without changing the proof. After n-1 passes, one record remains. The outer invariant forces that record to be no smaller than every key in the prefix, establishing sortedness. Every swap preserves the collection of records, and finite loop bounds establish termination.

Why the globally-smallest clause matters

An ordered prefix alone is insufficient. For example, [4,5 | 1,2] has a sorted prefix but it cannot be considered final. In selection sort, the invariant rules this state out because the prefix keys exceed suffix keys. Insertion sort is allowed to have that kind of relationship while it is still processing later records.

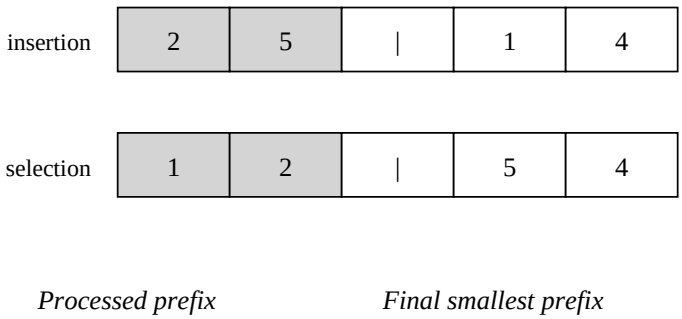


Figure S3.3 Selection finalizes globally smallest keys; the suffix need not be ordered.

New explanatory material; original source: CLRS Exercise 2.2-2; original worked proof

S3.4 Selection sort: exact comparisons and limited writes

Comparison count independent of input order

Pass i inspects n-i-1 other candidates. For i=0 through n-2, the total is (n-1)+(n-2)+...+1=n(n-1)/2. Already sorted, reversed, and all-equal inputs have the same comparison count. Thus best, average, and worst running times are Theta(n^2) under the usual constant-cost model for this standard implementation.

At most one nontrivial swap per pass

With the m!=i guard, there are at most n-1 nontrivial swaps and at most 2(n-1) array-slot writes from those swaps. This is an upper bound, not an exact count for every input. An already sorted array needs zero such swaps. A version that self-swaps makes exactly n-1 swap calls, which is a different counting convention.

Few swaps do not imply a small number of comparisons. A reverse array of eight distinct keys still needs 28 comparisons even though the minimum exchange mechanism can fix more than one position per swap. Never substitute the bubble-sort inversion swap formula for selection's distant exchanges.

When writes are expensive

A cost expression can be written as C_cmp * n(n-1)/2 + C_swap * s + lower-order overhead, where 0<=s<=n-1. If moving a record is expensive relative to comparing its key, the linear swap bound can matter. Sorting pointers may further reduce payload movement, although that approach changes representation and may require extra storage.

Selection is not automatically the best write-minimizing algorithm among all possibilities. The bound here compares a particular standard method with the other elementary methods in this supplement; it is not a universal lower bound on writes.

n	4	8	16	32
comparisons	6	28	120	496
max swaps	3	7	15	31

Figure S3.4 Quadratic comparisons coexist with a linear upper bound on swaps.

New explanatory material; original source: CLRS Exercise 2.2-2; original cost derivation

S3.5 Selection sort: why the usual exchange is unstable

A minimal counterexample

Use records (2,a), (2,b), (1,c), sorted by the numeric field. The first pass chooses (1,c) as the minimum and exchanges it with (2,a). The array becomes (1,c), (2,b), (2,a). The next pass sees two equal keys and makes no strict-smaller update. The final output places b before a, reversing their original order.

The output remains numerically sorted and preserves all identities. It is the stability requirement, and only that additional requirement, that fails. Using plain [2,2,1] hides the failure because the two equal numbers cannot be distinguished.

Why first-minimum selection is not enough

The inner loop with strict < already selects the first occurrence of a minimum. The instability above happens because a nonminimum record is displaced across an equal-key record by the long-distance swap. Changing only how equal minima are chosen cannot eliminate all such crossings.

Choosing the last minimum with <= can create other instability. For an all-equal array, it may exchange a boundary record with a later equal record. Both the selection rule and the movement operation have to be examined.

Restore a stable result deliberately

One approach is a stable extraction-and-shift selection variant. Another is to compare each record using (key, original_position), imposing an explicit secondary key. The latter simulates stable ordering but requires access to original positions and changes the comparator. It does not establish inherent stability of ordinary selection sort.

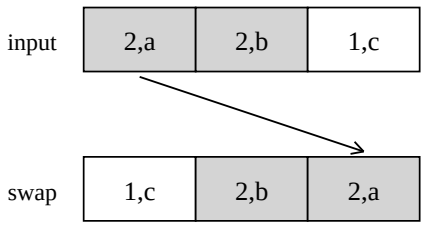


Figure S3.5 A nonminimum record crosses an equal-key record during a distant swap.

New explanatory material; original source: CLRS Exercise 2.2-2; original labeled counterexample

S3.6 Selection sort: a stable extraction-and-shift variant

Change the movement, not just the tie test

Find the first minimum in the active suffix using strict <. Save that complete record. Shift positions i through m-1 one place to the right, preserving their relative order, then put the saved minimum at i. This removes the endpoint swap that caused instability.

```
STABLE-SELECTION(A)
  for i = 0 to length(A)-2
    m = i
    for j = i+1 to length(A)-1
      if key(A[j]) < key(A[m])
        m = j
    if m != i
      saved = A[m]
      for j = m downto i+1
        A[j] = A[j-1]
      A[i] = saved
```

Why equal records retain order

The first minimum precedes every other equal minimum still active. Placing it at the boundary does not cross an earlier equal minimum. The shift moves every intervening record together and does not reverse their mutual order. Repeating this reasoning establishes stable extraction for every key group.

For (2,a), (2,b), (1,c), save c and shift a and b right. The result is (1,c), (2,a), (2,b), unlike the ordinary-swap result. The globally-smallest-prefix proof for selection still applies, with permutation preservation supplied by extraction plus shifting.

The tradeoff

Auxiliary storage remains O(1) records. The minimum searches still perform n(n-1)/2 comparisons. However, shifting can require Theta(n^2) array writes in the worst case. Thus the ordinary selection sort's linear bound on nontrivial swaps does not translate into a linear movement bound for this stable variant.

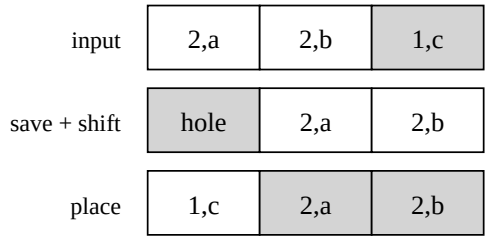


Figure S3.6 Extracting a minimum and shifting the block preserves equal-key order.

New explanatory material; original source: Original variant derived from CLRS Exercise 2.2-2

S3.7 Selection sort: implementation, boundaries, and checks

In-place ordinary selection

The guard avoids self-swaps and makes the write convention explicit. This routine is correct for comparable keys but does not promise stable tie order.

```
def selection_sort(a, key=lambda x: x):
    n = len(a)
    for i in range(n-1):
        m = i
        for j in range(i+1, n):
            if key(a[j]) < key(a[m]):
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]
    return a
```

Common failures

Reset m for each outer pass. Search only the unresolved suffix, including its final position. Do not place a swap inside every candidate comparison and still describe the result as the one-swap selection algorithm; that mutation is a different procedure with a different movement count.

The array can be empty: range(-1) produces no iterations in this Python listing. A singleton similarly needs no pass. For descending order, update m when the candidate key is strictly larger. For record sorting, exchange whole records and do not rewrite only their keys.

Checks that match the contract

For [5,1,4,2,8], expect [1,2,4,5,8], ten comparisons, and two guarded swaps. For labeled [(2,a), (2,b), (1,c)], expect a numerically ordered output but do not assert stability for this version. If stable behavior is required, use and test the extraction-and-shift variant instead.

Preservation should compare record identities or full multiplicities, not just sets of keys. The algorithm's n-1 swap upper bound also supplies an instrumentation check for the guarded implementation.

New explanatory material; original source: CLRS Exercise 2.2-2; original Python implementation

S3.8 Selection sort: deep questions and neighboring concepts

Why must the suffix be scanned even if the prefix is ordered?

An ordered prefix is already finalized by the invariant, but the next smallest key can be anywhere in the suffix. Without extra information about key values or their arrangement, every candidate can affect which record must be selected. The fixed scan therefore performs the same number of comparisons on ordered and disordered input.

Why do n-1 passes suffice?

After n-1 passes, the prefix contains the n-1 smallest records in order, every remaining key is no smaller than the prefix, and exactly one record remains. That record necessarily occupies the final position. A last minimum search on a singleton provides no new information.

Selection sort versus selecting one rank

The name selection sort does not mean an optimal selection algorithm for the kth smallest element. Sorting the entire array is more work than needed if only one rank is requested. After k passes, ordinary selection has finalized the k smallest records, but this partial method costs O(kn) comparisons and is not a general linear-time rank-selection guarantee.

Selection versus insertion

Selection finds an extremum in the unresolved suffix and finalizes a boundary record. Insertion takes the next record from the original processing order and inserts it into a sorted processed prefix. A selection prefix is globally smallest; an insertion prefix need not be. Both can use constant auxiliary storage, yet their adaptivity and stability rules differ.

Selection versus bubble

One long-distance selection swap may remove many inversions and may cross equal-key identities. Each strict adjacent bubble swap removes exactly one inversion and preserves equal-key order. An answer based only on "both move the minimum" overlooks these consequences.

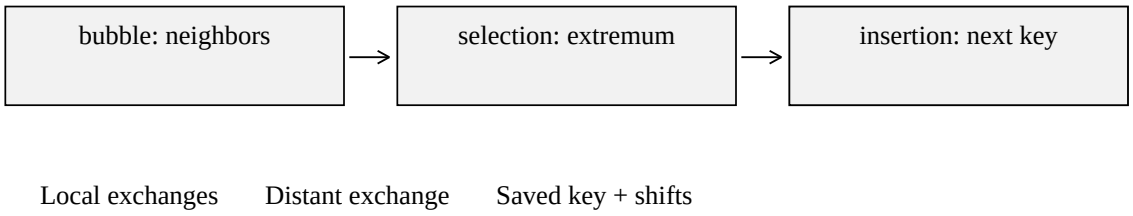


Figure S3.8 Similar final boundaries arise from different movement mechanisms.

New explanatory material; original source: CLRS Exercise 2.2-2; original answer bank

2 Getting Started

This chapter will familiarize you with the framework we shall use throughout the book to think about the design and analysis of algorithms. It is self-contained, but it does include several references to material that we introduce in Chapters 3 and 4. (It also contains several summations, which Appendix A shows how to solve.)

We begin by examining the insertion sort algorithm to solve the sorting problem introduced in Chapter 1. We define a "pseudocode" that should be familiar to you if you have done computer programming, and we use it to show how we shall specify our algorithms. Having specified the insertion sort algorithm, we then argue that it correctly sorts, and we analyze its running time. The analysis introduces a notation that focuses on how that time increases with the number of items to be sorted. Following our discussion of insertion sort, we introduce the divide-and-conquer approach to the design of algorithms and use it to develop an algorithm called merge sort. We end with an analysis of merge sort's running time.

2.1 Insertion sort

Our first algorithm, insertion sort, solves the ***sorting problem*** introduced in Chapter 1:

Input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

The numbers that we wish to sort are also known as the ***keys***. Although conceptually we are sorting a sequence, the input comes to us in the form of an array with n elements.

In this book, we shall typically describe algorithms as programs written in a ***pseudocode*** that is similar in many respects to C, C++, Java, Python, or Pascal. If you have been introduced to any of these languages, you should have little trouble

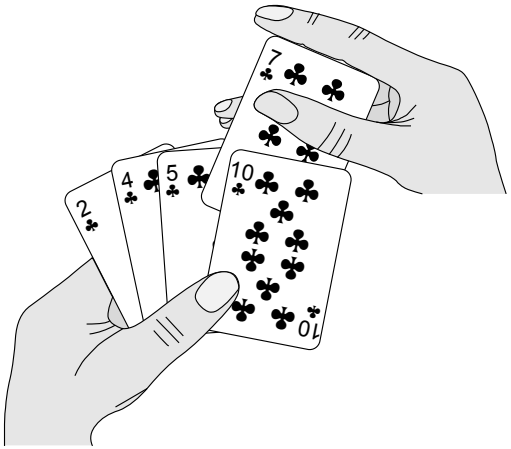


Figure 2.1 Sorting a hand of cards using insertion sort.

reading our algorithms. What separates pseudocode from "real" code is that in pseudocode, we employ whatever expressive method is most clear and concise to specify a given algorithm. Sometimes, the clearest method is English, so do not be surprised if you come across an English phrase or sentence embedded within a section of "real" code. Another difference between pseudocode and real code is that pseudocode is not typically concerned with issues of software engineering. Issues of data abstraction, modularity, and error handling are often ignored in order to convey the essence of the algorithm more concisely.

We start with ***insertion sort***, which is an efficient algorithm for sorting a small number of elements. Insertion sort works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table. We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards already in the hand, from right to left, as illustrated in Figure 2.1. At all times, the cards held in the left hand are sorted, and these cards were originally the top cards of the pile on the table.

We present our pseudocode for insertion sort as a procedure called INSERTION-SORT, which takes as a parameter an array $A[1..n]$ containing a sequence of length n that is to be sorted. (In the code, the number n of elements in A is denoted by $A.length$.) The algorithm sorts the input numbers ***in place***: it rearranges the numbers within the array A, with at most a constant number of them stored outside the array at any time. The input array A contains the sorted output sequence when the INSERTION-SORT procedure is finished.

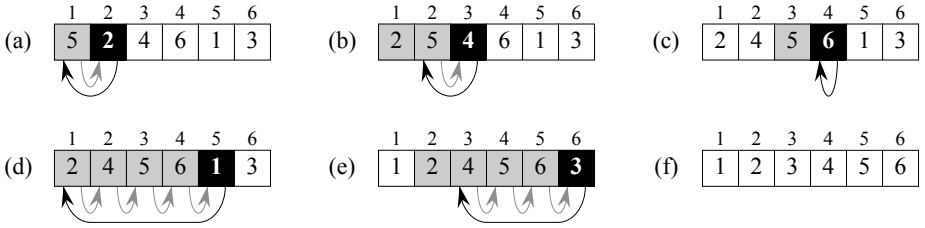


Figure 2.2 The operation of INSERTION-SORT on the array $A = \langle 5, 2, 4, 6, 1, 3 \rangle$. Array indices appear above the rectangles, and values stored in the array positions appear within the rectangles. **(a)–(e)** The iterations of the **for** loop of lines 1–8. In each iteration, the black rectangle holds the key taken from $A[j]$, which is compared with the values in shaded rectangles to its left in the test of line 5. Shaded arrows show array values moved one position to the right in line 6, and black arrows indicate where the key moves to in line 8. **(f)** The final sorted array.

```
INSERTION-SORT(A)
1  for j = 2 to A.length
2      key = A[j]
3      // Insert A[j] into the sorted sequence A[1 .. j − 1].
4      i = j − 1
5      while i > 0 and A[i] > key
6          A[i + 1] = A[i]
7          i = i − 1
8      A[i + 1] = key
```

Loop invariants and the correctness of insertion sort

Figure 2.2 shows how this algorithm works for $A = \langle 5, 2, 4, 6, 1, 3 \rangle$. The index j indicates the "current card" being inserted into the hand. At the beginning of each iteration of the **for** loop, which is indexed by j, the subarray consisting of elements $A[1 \,..\, j-1]$ constitutes the currently sorted hand, and the remaining subarray $A[j+1 \,..\, n]$ corresponds to the pile of cards still on the table. In fact, elements $A[1 \,..\, j-1]$ are the elements *originally* in positions 1 through $j-1$, but now in sorted order. We state these properties of $A[1 \,..\, j-1]$ formally as a ***loop invariant***:

> At the start of each iteration of the **for** loop of lines 1–8, the subarray $A[1 \,..\, j-1]$ consists of the elements originally in $A[1 \,..\, j-1]$, but in sorted order.

We use loop invariants to help us understand why an algorithm is correct. We must show three things about a loop invariant:

Initialization: It is true prior to the first iteration of the loop.

Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration.

Termination: When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

When the first two properties hold, the loop invariant is true prior to every iteration of the loop. (Of course, we are free to use established facts other than the loop invariant itself to prove that the loop invariant remains true before each iteration.) Note the similarity to mathematical induction, where to prove that a property holds, you prove a base case and an inductive step. Here, showing that the invariant holds before the first iteration corresponds to the base case, and showing that the invariant holds from iteration to iteration corresponds to the inductive step.

The third property is perhaps the most important one, since we are using the loop invariant to show correctness. Typically, we use the loop invariant along with the condition that caused the loop to terminate. The termination property differs from how we usually use mathematical induction, in which we apply the inductive step infinitely; here, we stop the "induction" when the loop terminates.

Let us see how these properties hold for insertion sort.

Initialization: We start by showing that the loop invariant holds before the first loop iteration, when $j = 2$.[1] The subarray $A[1 \,..\, j-1]$, therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$. Moreover, this subarray is sorted (trivially, of course), which shows that the loop invariant holds prior to the first iteration of the loop.

Maintenance: Next, we tackle the second property: showing that each iteration maintains the loop invariant. Informally, the body of the **for** loop works by moving $A[j-1]$, $A[j-2]$, $A[j-3]$, and so on by one position to the right until it finds the proper position for $A[j]$ (lines 4–7), at which point it inserts the value of $A[j]$ (line 8). The subarray $A[1 \,..\, j]$ then consists of the elements originally in $A[1 \,..\, j]$, but in sorted order. Incrementing j for the next iteration of the **for** loop then preserves the loop invariant.

A more formal treatment of the second property would require us to state and show a loop invariant for the **while** loop of lines 5–7. At this point, however,

---

[1] When the loop is a **for** loop, the moment at which we check the loop invariant just prior to the first iteration is immediately after the initial assignment to the loop-counter variable and just before the first test in the loop header. In the case of INSERTION-SORT, this time is after assigning 2 to the variable j but before the first test of whether $j \leq A.length$.

we prefer not to get bogged down in such formalism, and so we rely on our informal analysis to show that the second property holds for the outer loop.

Termination: Finally, we examine what happens when the loop terminates. The condition causing the **for** loop to terminate is that $j > A.length = n$. Because each loop iteration increases j by 1, we must have $j = n + 1$ at that time. Substituting $n + 1$ for j in the wording of loop invariant, we have that the subarray $A[1..n]$ consists of the elements originally in $A[1..n]$, but in sorted order. Observing that the subarray $A[1..n]$ is the entire array, we conclude that the entire array is sorted. Hence, the algorithm is correct.

We shall use this method of loop invariants to show correctness later in this chapter and in other chapters as well.

Pseudocode conventions

We use the following conventions in our pseudocode.

- Indentation indicates block structure. For example, the body of the **for** loop that begins on line 1 consists of lines 2–8, and the body of the **while** loop that begins on line 5 contains lines 6–7 but not line 8. Our indentation style applies to **if-else** statements[2] as well. Using indentation instead of conventional indicators of block structure, such as **begin** and **end** statements, greatly reduces clutter while preserving, or even enhancing, clarity.[3]
- The looping constructs **while**, **for**, and **repeat-until** and the **if-else** conditional construct have interpretations similar to those in C, C++, Java, Python, and Pascal.[4] In this book, the loop counter retains its value after exiting the loop, unlike some situations that arise in C++, Java, and Pascal. Thus, immediately after a **for** loop, the loop counter's value is the value that first exceeded the **for** loop bound. We used this property in our correctness argument for insertion sort. The **for** loop header in line 1 is **for** $j = 2$ **to** $A.length$, and so when this loop terminates, $j = A.length + 1$ (or, equivalently, $j = n + 1$, since $n = A.length$). We use the keyword **to** when a **for** loop increments its loop

---

[2] In an **if-else** statement, we indent **else** at the same level as its matching **if**. Although we omit the keyword **then**, we occasionally refer to the portion executed when the test following **if** is true as a ***then clause***. For multiway tests, we use **elseif** for tests after the first one.

[3] Each pseudocode procedure in this book appears on one page so that you will not have to discern levels of indentation in code that is split across pages.

[4] Most block-structured languages have equivalent constructs, though the exact syntax may differ. Python lacks **repeat-until** loops, and its **for** loops operate a little differently from the **for** loops in this book.

counter in each iteration, and we use the keyword **downto** when a **for** loop decrements its loop counter. When the loop counter changes by an amount greater than 1, the amount of change follows the optional keyword **by**.

- The symbol "//" indicates that the remainder of the line is a comment.
- A multiple assignment of the form $i = j = e$ assigns to both variables i and j the value of expression e; it should be treated as equivalent to the assignment $j = e$ followed by the assignment $i = j$.
- Variables (such as i, j, and *key*) are local to the given procedure. We shall not use global variables without explicit indication.
- We access array elements by specifying the array name followed by the index in square brackets. For example, $A[i]$ indicates the ith element of the array A. The notation ".." is used to indicate a range of values within an array. Thus, $A[1 \,..\, j]$ indicates the subarray of A consisting of the j elements $A[1], A[2], \ldots, A[j]$.
- We typically organize compound data into ***objects***, which are composed of ***attributes***. We access a particular attribute using the syntax found in many object-oriented programming languages: the object name, followed by a dot, followed by the attribute name. For example, we treat an array as an object with the attribute *length* indicating how many elements it contains. To specify the number of elements in an array A, we write $A.length$.

  We treat a variable representing an array or object as a pointer to the data representing the array or object. For all attributes f of an object x, setting $y = x$ causes $y.f$ to equal $x.f$. Moreover, if we now set $x.f = 3$, then afterward not only does $x.f$ equal 3, but $y.f$ equals 3 as well. In other words, x and y point to the same object after the assignment $y = x$.

  Our attribute notation can "cascade." For example, suppose that the attribute f is itself a pointer to some type of object that has an attribute g. Then the notation $x.f.g$ is implicitly parenthesized as $(x.f).g$. In other words, if we had assigned $y = x.f$, then $x.f.g$ is the same as $y.g$.

  Sometimes, a pointer will refer to no object at all. In this case, we give it the special value NIL.
- We pass parameters to a procedure ***by value***: the called procedure receives its own copy of the parameters, and if it assigns a value to a parameter, the change is *not* seen by the calling procedure. When objects are passed, the pointer to the data representing the object is copied, but the object's attributes are not. For example, if x is a parameter of a called procedure, the assignment $x = y$ within the called procedure is not visible to the calling procedure. The assignment $x.f = 3$, however, is visible. Similarly, arrays are passed by pointer, so that

a pointer to the array is passed, rather than the entire array, and changes to individual array elements are visible to the calling procedure.

- A **return** statement immediately transfers control back to the point of call in the calling procedure. Most **return** statements also take a value to pass back to the caller. Our pseudocode differs from many programming languages in that we allow multiple values to be returned in a single **return** statement.
- The boolean operators "and" and "or" are ***short circuiting***. That is, when we evaluate the expression "x and y" we first evaluate x. If x evaluates to FALSE, then the entire expression cannot evaluate to TRUE, and so we do not evaluate y. If, on the other hand, x evaluates to TRUE, we must evaluate y to determine the value of the entire expression. Similarly, in the expression "x or y" we evaluate the expression y only if x evaluates to FALSE. Short-circuiting operators allow us to write boolean expressions such as "$x \neq$ NIL and $x.f = y$" without worrying about what happens when we try to evaluate $x.f$ when x is NIL.
- The keyword **error** indicates that an error occurred because conditions were wrong for the procedure to have been called. The calling procedure is responsible for handling the error, and so we do not specify what action to take.

Exercises

2.1-1
Using Figure 2.2 as a model, illustrate the operation of INSERTION-SORT on the array $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.

2.1-2
Rewrite the INSERTION-SORT procedure to sort into nonincreasing instead of nondecreasing order.

2.1-3
Consider the ***searching problem***:

Input: A sequence of n numbers $A = \langle a_1, a_2, \ldots, a_n \rangle$ and a value v.

Output: An index i such that $v = A[i]$ or the special value NIL if v does not appear in A.

Write pseudocode for ***linear search***, which scans through the sequence, looking for v. Using a loop invariant, prove that your algorithm is correct. Make sure that your loop invariant fulfills the three necessary properties.

2.1-4
Consider the problem of adding two n-bit binary integers, stored in two n-element arrays A and B. The sum of the two integers should be stored in binary form in

an $(n + 1)$-element array C. State the problem formally and write pseudocode for adding the two integers.

2.2 Analyzing algorithms

Analyzing an algorithm has come to mean predicting the resources that the algorithm requires. Occasionally, resources such as memory, communication bandwidth, or computer hardware are of primary concern, but most often it is computational time that we want to measure. Generally, by analyzing several candidate algorithms for a problem, we can identify a most efficient one. Such analysis may indicate more than one viable candidate, but we can often discard several inferior algorithms in the process.

Before we can analyze an algorithm, we must have a model of the implementation technology that we will use, including a model for the resources of that technology and their costs. For most of this book, we shall assume a generic one-processor, ***random-access machine (RAM)*** model of computation as our implementation technology and understand that our algorithms will be implemented as computer programs. In the RAM model, instructions are executed one after another, with no concurrent operations.

Strictly speaking, we should precisely define the instructions of the RAM model and their costs. To do so, however, would be tedious and would yield little insight into algorithm design and analysis. Yet we must be careful not to abuse the RAM model. For example, what if a RAM had an instruction that sorts? Then we could sort in just one instruction. Such a RAM would be unrealistic, since real computers do not have such instructions. Our guide, therefore, is how real computers are designed. The RAM model contains instructions commonly found in real computers: arithmetic (such as add, subtract, multiply, divide, remainder, floor, ceiling), data movement (load, store, copy), and control (conditional and unconditional branch, subroutine call and return). Each such instruction takes a constant amount of time.

The data types in the RAM model are integer and floating point (for storing real numbers). Although we typically do not concern ourselves with precision in this book, in some applications precision is crucial. We also assume a limit on the size of each word of data. For example, when working with inputs of size n, we typically assume that integers are represented by $c \lg n$ bits for some constant $c \geq 1$. We require $c \geq 1$ so that each word can hold the value of n, enabling us to index the individual input elements, and we restrict c to be a constant so that the word size does not grow arbitrarily. (If the word size could grow arbitrarily, we could store huge amounts of data in one word and operate on it all in constant time—clearly an unrealistic scenario.)

Real computers contain instructions not listed above, and such instructions represent a gray area in the RAM model. For example, is exponentiation a constant-time instruction? In the general case, no; it takes several instructions to compute x^y when x and y are real numbers. In restricted situations, however, exponentiation is a constant-time operation. Many computers have a "shift left" instruction, which in constant time shifts the bits of an integer by k positions to the left. In most computers, shifting the bits of an integer by one position to the left is equivalent to multiplication by 2, so that shifting the bits by k positions to the left is equivalent to multiplication by 2^k. Therefore, such computers can compute 2^k in one constant-time instruction by shifting the integer 1 by k positions to the left, as long as k is no more than the number of bits in a computer word. We will endeavor to avoid such gray areas in the RAM model, but we will treat computation of 2^k as a constant-time operation when k is a small enough positive integer.

In the RAM model, we do not attempt to model the memory hierarchy that is common in contemporary computers. That is, we do not model caches or virtual memory. Several computational models attempt to account for memory-hierarchy effects, which are sometimes significant in real programs on real machines. A handful of problems in this book examine memory-hierarchy effects, but for the most part, the analyses in this book will not consider them. Models that include the memory hierarchy are quite a bit more complex than the RAM model, and so they can be difficult to work with. Moreover, RAM-model analyses are usually excellent predictors of performance on actual machines.

Analyzing even a simple algorithm in the RAM model can be a challenge. The mathematical tools required may include combinatorics, probability theory, algebraic dexterity, and the ability to identify the most significant terms in a formula. Because the behavior of an algorithm may be different for each possible input, we need a means for summarizing that behavior in simple, easily understood formulas.

Even though we typically select only one machine model to analyze a given algorithm, we still face many choices in deciding how to express our analysis. We would like a way that is simple to write and manipulate, shows the important characteristics of an algorithm's resource requirements, and suppresses tedious details.

Analysis of insertion sort

The time taken by the INSERTION-SORT procedure depends on the input: sorting a thousand numbers takes longer than sorting three numbers. Moreover, INSERTION-SORT can take different amounts of time to sort two input sequences of the same size depending on how nearly sorted they already are. In general, the time taken by an algorithm grows with the size of the input, so it is traditional to describe the running time of a program as a function of the size of its input. To do so, we need to define the terms "running time" and "size of input" more carefully.

The best notion for ***input size*** depends on the problem being studied. For many problems, such as sorting or computing discrete Fourier transforms, the most natural measure is the *number of items in the input*—for example, the array size n for sorting. For many other problems, such as multiplying two integers, the best measure of input size is the *total number of bits* needed to represent the input in ordinary binary notation. Sometimes, it is more appropriate to describe the size of the input with two numbers rather than one. For instance, if the input to an algorithm is a graph, the input size can be described by the numbers of vertices and edges in the graph. We shall indicate which input size measure is being used with each problem we study.

The ***running time*** of an algorithm on a particular input is the number of primitive operations or "steps" executed. It is convenient to define the notion of step so that it is as machine-independent as possible. For the moment, let us adopt the following view. A constant amount of time is required to execute each line of our pseudocode. One line may take a different amount of time than another line, but we shall assume that each execution of the ith line takes time c_i, where c_i is a constant. This viewpoint is in keeping with the RAM model, and it also reflects how the pseudocode would be implemented on most actual computers.[5]

In the following discussion, our expression for the running time of INSERTION-SORT will evolve from a messy formula that uses all the statement costs c_i to a much simpler notation that is more concise and more easily manipulated. This simpler notation will also make it easy to determine whether one algorithm is more efficient than another.

We start by presenting the INSERTION-SORT procedure with the time "cost" of each statement and the number of times each statement is executed. For each $j = 2, 3, \ldots, n$, where $n = A.length$, we let t_j denote the number of times the **while** loop test in line 5 is executed for that value of j. When a **for** or **while** loop exits in the usual way (i.e., due to the test in the loop header), the test is executed one time more than the loop body. We assume that comments are not executable statements, and so they take no time.

---

[5]There are some subtleties here. Computational steps that we specify in English are often variants of a procedure that requires more than just a constant amount of time. For example, later in this book we might say "sort the points by x-coordinate," which, as we shall see, takes more than a constant amount of time. Also, note that a statement that calls a subroutine takes constant time, though the subroutine, once invoked, may take more. That is, we separate the process of ***calling*** the subroutine—passing parameters to it, etc.—from the process of ***executing*** the subroutine.

INSERTION-SORT(A)	*cost*	*times*
1 **for** $j = 2$ **to** $A.length$	c_1	n
2 $key = A[j]$	c_2	$n-1$
3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4 $i = j-1$	c_4	$n-1$
5 **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n}(t_j-1)$
7 $i = i-1$	c_7	$\sum_{j=2}^{n}(t_j-1)$
8 $A[i+1] = key$	c_8	$n-1$

The running time of the algorithm is the sum of running times for each statement executed; a statement that takes c_i steps to execute and executes n times will contribute $c_i n$ to the total running time.[6] To compute $T(n)$, the running time of INSERTION-SORT on an input of n values, we sum the products of the *cost* and *times* columns, obtaining

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) \\ &\quad + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1) \; . \end{aligned}$$

Even for inputs of a given size, an algorithm's running time may depend on *which* input of that size is given. For example, in INSERTION-SORT, the best case occurs if the array is already sorted. For each $j = 2, 3, \ldots, n$, we then find that $A[i] \le key$ in line 5 when i has its initial value of $j-1$. Thus $t_j = 1$ for $j = 2, 3, \ldots, n$, and the best-case running time is

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \; . \end{aligned}$$

We can express this running time as $an + b$ for *constants* a and b that depend on the statement costs c_i; it is thus a ***linear function*** of n.

If the array is in reverse sorted order—that is, in decreasing order—the worst case results. We must compare each element $A[j]$ with each element in the entire sorted subarray $A[1..j-1]$, and so $t_j = j$ for $j = 2, 3, \ldots, n$. Noting that

[6] This characteristic does not necessarily hold for a resource such as memory. A statement that references m words of memory and is executed n times does not necessarily reference mn distinct words of memory.

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1$$

and

$$\sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$$

(see Appendix A for a review of how to solve these summations), we find that in the worst case, the running time of INSERTION-SORT is

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8) \, . \end{aligned}$$

We can express this worst-case running time as $an^2 + bn + c$ for constants a, b, and c that again depend on the statement costs c_i; it is thus a ***quadratic function*** of n.

Typically, as in insertion sort, the running time of an algorithm is fixed for a given input, although in later chapters we shall see some interesting "randomized" algorithms whose behavior can vary even for a fixed input.

Worst-case and average-case analysis

In our analysis of insertion sort, we looked at both the best case, in which the input array was already sorted, and the worst case, in which the input array was reverse sorted. For the remainder of this book, though, we shall usually concentrate on finding only the ***worst-case running time***, that is, the longest running time for *any* input of size n. We give three reasons for this orientation.

- The worst-case running time of an algorithm gives us an upper bound on the running time for any input. Knowing it provides a guarantee that the algorithm will never take any longer. We need not make some educated guess about the running time and hope that it never gets much worse.
- For some algorithms, the worst case occurs fairly often. For example, in searching a database for a particular piece of information, the searching algorithm's worst case will often occur when the information is not present in the database. In some applications, searches for absent information may be frequent.

- The "average case" is often roughly as bad as the worst case. Suppose that we randomly choose n numbers and apply insertion sort. How long does it take to determine where in subarray $A[1 \,..\, j-1]$ to insert element $A[j]$? On average, half the elements in $A[1 \,..\, j-1]$ are less than $A[j]$, and half the elements are greater. On average, therefore, we check half of the subarray $A[1 \,..\, j-1]$, and so t_j is about $j/2$. The resulting average-case running time turns out to be a quadratic function of the input size, just like the worst-case running time.

In some particular cases, we shall be interested in the ***average-case*** running time of an algorithm; we shall see the technique of ***probabilistic analysis*** applied to various algorithms throughout this book. The scope of average-case analysis is limited, because it may not be apparent what constitutes an "average" input for a particular problem. Often, we shall assume that all inputs of a given size are equally likely. In practice, this assumption may be violated, but we can sometimes use a ***randomized algorithm***, which makes random choices, to allow a probabilistic analysis and yield an ***expected*** running time. We explore randomized algorithms more in Chapter 5 and in several other subsequent chapters.

Order of growth

We used some simplifying abstractions to ease our analysis of the INSERTION-SORT procedure. First, we ignored the actual cost of each statement, using the constants c_i to represent these costs. Then, we observed that even these constants give us more detail than we really need: we expressed the worst-case running time as $an^2 + bn + c$ for some constants a, b, and c that depend on the statement costs c_i. We thus ignored not only the actual statement costs, but also the abstract costs c_i.

We shall now make one more simplifying abstraction: it is the ***rate of growth***, or ***order of growth***, of the running time that really interests us. We therefore consider only the leading term of a formula (e.g., an^2), since the lower-order terms are relatively insignificant for large values of n. We also ignore the leading term's constant coefficient, since constant factors are less significant than the rate of growth in determining computational efficiency for large inputs. For insertion sort, when we ignore the lower-order terms and the leading term's constant coefficient, we are left with the factor of n^2 from the leading term. We write that insertion sort has a worst-case running time of $\Theta(n^2)$ (pronounced "theta of n-squared"). We shall use Θ-notation informally in this chapter, and we will define it precisely in Chapter 3.

We usually consider one algorithm to be more efficient than another if its worst-case running time has a lower order of growth. Due to constant factors and lower-order terms, an algorithm whose running time has a higher order of growth might take less time for small inputs than an algorithm whose running time has a lower

S4.1 Insertion sort: the saved key and the temporary hole

Processed order rather than global selection

Insertion sort takes the next input record, saves it, and inserts it into the ordered processed prefix. It shifts larger records one position to the right until the saved key can be placed after every key no greater than it. The untouched suffix contains records not yet processed.

The saved record is essential because the first shift overwrites its original slot. During shifting, the array may temporarily contain a duplicate of a shifted record and omit the saved record; the saved variable accounts for the missing identity. Preservation should be stated over the array together with this saved record and the temporary hole, not naively over every intermediate array state.

One insertion in detail

For [2,4,5 | 1,3], save 1. Shift 5 from position 2 to 3, shift 4 from 1 to 2, and shift 2 from 0 to 1. Write saved 1 at position 0, giving [1,2,4,5 | 3]. A single insertion can move a key across the whole prefix.

Zero-based procedure

```
INSERTION(A)
  for j = 1 to length(A)-1
    saved = A[j]
    i = j-1
    while i >= 0 and key(A[i]) > key(saved)
      A[i+1] = A[i]
      i = i-1
    A[i+1] = saved
```

The boundary check must be evaluated before reading A[i] when i becomes negative. Strict > keeps an existing equal-key record before the newly inserted one and therefore gives stability.



Figure S4.1 Save the record before larger prefix records overwrite its slot.

New explanatory material; original source: CLRS 2.1; original supplementary explanation

S4.2 Insertion sort: inversions and adaptive running time

A sharper parameter than input length alone

Let I be the number of strict inversions in the initial array. In the shift-based insertion routine, each larger predecessor shifted past the saved record corresponds to one inversion involving that record. Every inversion is resolved exactly once during processing. The total number of shifts is therefore I.

There are n-1 outer iterations. Each iteration has a constant amount of work plus its shifts and associated key comparisons. Consequently running time is Theta(n+I) for n>=1 under the constant-cost model, with the empty case handled separately. The key-comparison count is at least I and at most I+n-1; a final unsuccessful key comparison may or may not occur, depending on whether the left boundary is reached.

Important cases

An already ordered input has I=0 and takes Theta(n) time. Distinct reverse order has I=n(n-1)/2 and takes Theta(n^2). Under a uniform random permutation of distinct keys, E[I]=n(n-1)/4, producing expected quadratic time.

For [2,3,...,n,1], there are n-1 inversions and one insertion shifts the whole preceding prefix, so the total is Theta(n). For [n,1,2,...,n-1], each of the following records crosses the initial maximum once; I is also n-1 and total time is linear. This is a stronger adaptivity guarantee than a vague claim about "nearly sorted" data.

Counts are not identical

Each shift is one array write. The listing also writes the saved record once per outer iteration, giving I+(n-1) array-slot writes, even if the saved record is written back to its unchanged position. A guarded optimization can reduce some writes but changes the exact count.

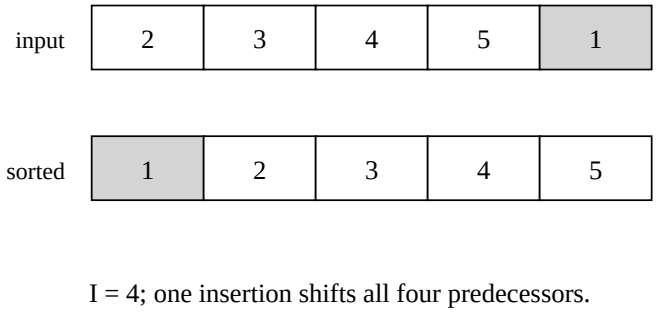


Figure S4.2 Inversion count measures pair disorder, not the number of misplaced slots.

New explanatory material; original source: CLRS 2.2 and Problem 2-4; original derivation

S4.3 Insertion sort: ties, binary search, and data structures

Stable placement among equals

Shift a predecessor only if its key is strictly greater than the saved key. Thus the new record is placed after the existing equal-key records. Replacing > with >= moves old equal records right and can reverse tie order. A descending stable version reverses the strict direction but retains the same no-crossing rule for equals.

Binary insertion separates search from movement

The insertion location can be found by binary search in the ordered prefix. To preserve stability, choose the position after all existing equal keys, often called an upper-bound position. Finding the location costs O(log j) comparisons for a prefix of size j, but shifting the intervening array records can still take O(j) writes.

Over all insertions, binary search can reduce worst-case comparisons to O(n log n) while total movement remains Theta(n^2) in reverse order. Binary insertion therefore does not turn a standard contiguous-array insertion sorter into an O(n log n) total-time algorithm under a model counting each moved record.

Arrays and linked lists

An array supports constant-time indexed access, but inserting into its middle requires moving records. A linked list may insert at a known node position by changing a constant number of links, yet locating that position can still require a linear traversal. Binary search is not automatically logarithmic time on a singly linked list, because accessing the middle is not constant time.

An incremental insertion sorter can process arriving records into an ordered collection. For an array, that still entails shifting and capacity management. Calling the method online does not remove its worst-case cost per insertion.

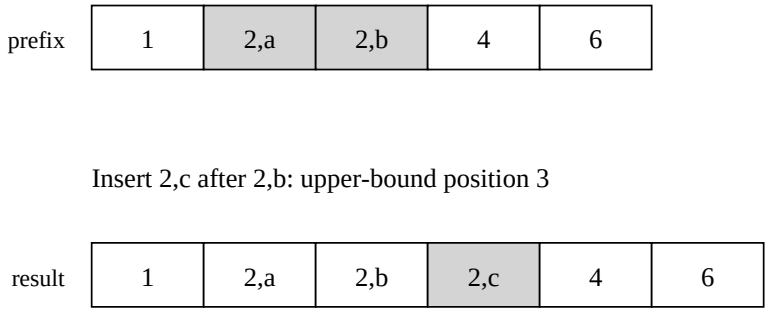


Figure S4.3 Faster location search does not eliminate shifting in an array.

New explanatory material; original source: CLRS 2.1 and Exercise 2.3-6; original supplementary analysis

S4.4 Insertion sort: implementation and proof obligations

In-place stable implementation

```
def insertion_sort(a, key=lambda x: x):
    for j in range(1, len(a)):
        saved = a[j]
        saved_key = key(saved)
        i = j-1
        while i >= 0 and key(a[i]) > saved_key:
            a[i+1] = a[i]
            i -= 1
        a[i+1] = saved
    return a
```

Outer correctness argument

Before iteration j, the prefix a[0..j-1] contains exactly the first j original records, in stable sorted order. Initialization holds for a singleton prefix. The iteration saves the next record and moves only the suffix of larger prefix records. Placing saved at i+1 produces the ordered prefix including that record; strict comparison preserves earlier equal records before it.

The loop counter advances through a finite list. The inner counter decreases and is bounded below by -1, so termination holds. The saved record and shifted block restore the same collection of processed records after every completed insertion. At the end the entire input has been processed, proving sortedness, preservation, and stability.

Trace and failure checks

For [5,2,4,6,1,3], the completed prefixes grow as [2,5], [2,4,5], [2,4,5,6], [1,2,4,5,6], and [1,2,3,4,5,6]. There are nine strict inversions and nine shifts. Check the final saved-key placement even when no shifts occur; omitting it is dangerous after a nonempty shifting loop.

Boundary evaluation matters in Python: a[-1] is a valid access to the last element, so testing the key before i>=0 can produce a logically wrong comparison instead of an immediate bounds exception. Other languages may fail outright. The listed short-circuit order prevents that access.

S4.5 Insertion sort: deep questions and hybrid use

Why is insertion useful inside a merge-based algorithm?

Small subarrays can be sorted with insertion before merges combine them. For subarray size t, sorting approximately n/t blocks costs O(nt) in the worst case. Merging these blocks across about log2(n/t) levels costs O(n log(n/t)). With a fixed small t, the complete method retains an O(n log n) worst-case bound.

The practical threshold is not a universal constant. It depends on implementation, key costs, memory access, and overhead. A teaching example can explain the formula without presenting an unmeasured cutoff as a performance recommendation.

Does stable insertion repair an unstable merge?

No. Stability of the whole hybrid requires stability of the block sorts and every subsequent merge. If a merge takes from the right on equal keys, it can undo the tie order established inside and between blocks. Conversely, stable merge cannot repair arbitrary equal-key reversals already caused by an unstable block sort.

Why not use insertion for every large input?

Its adaptive Theta(n+I) behavior is excellent when I is small, but I can be quadratic. A worst-case guarantee must account for reverse or otherwise highly inverted data. A statement based on one ordered benchmark does not establish the bound for arbitrary inputs.

Question bank

"How is insertion different from bubble?" It saves one record and shifts the larger prefix segment; bubble repeatedly exchanges adjacent pairs in passes. "Does insertion know the global minimum?" Not before processing all records. "Why is its best case linear?" Every new key immediately satisfies the placement condition after a constant amount of work. "Can binary insertion beat the comparison lower bound?" No; its worst-case comparison cost is O(n log n), and it still has quadratic array movement in the worst case.

A proof-focused answer

For a proposed optimization, state the new invariant and re-count comparisons and writes separately. Keeping the same final output is not enough to justify a claimed running-time improvement, and matching a trace on one input is not enough to establish correctness on all valid inputs.

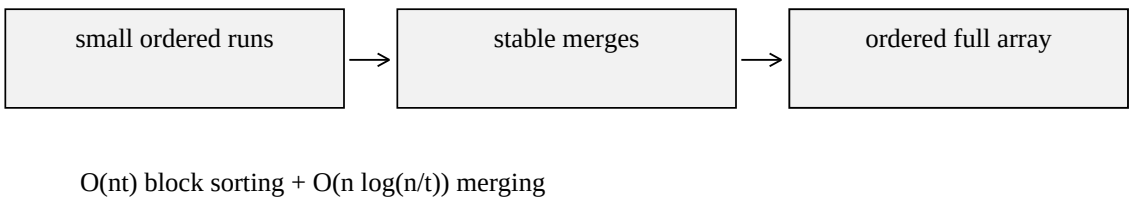


Figure S4.5 Stable small-block sorting plus stable merges forms a stable hybrid.

New explanatory material; original source: CLRS Problem 2-1 and 2.3; original supplementary synthesis

2.3.1 The divide-and-conquer approach

Many useful algorithms are ***recursive*** in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related subproblems. These algorithms typically follow a ***divide-and-conquer*** approach: they break the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems recursively, and then combine these solutions to create a solution to the original problem.

The divide-and-conquer paradigm involves three steps at each level of the recursion:

Divide the problem into a number of subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner.

Combine the solutions to the subproblems into the solution for the original problem.

The ***merge sort*** algorithm closely follows the divide-and-conquer paradigm. Intuitively, it operates as follows.

Divide: Divide the n-element sequence to be sorted into two subsequences of $n/2$ elements each.

Conquer: Sort the two subsequences recursively using merge sort.

Combine: Merge the two sorted subsequences to produce the sorted answer.

The recursion "bottoms out" when the sequence to be sorted has length 1, in which case there is no work to be done, since every sequence of length 1 is already in sorted order.

The key operation of the merge sort algorithm is the merging of two sorted sequences in the "combine" step. We merge by calling an auxiliary procedure MERGE(A, p, q, r), where A is an array and p, q, and r are indices into the array such that $p \le q < r$. The procedure assumes that the subarrays $A[p\,..\,q]$ and $A[q+1\,..\,r]$ are in sorted order. It ***merges*** them to form a single sorted subarray that replaces the current subarray $A[p\,..\,r]$.

Our MERGE procedure takes time $\Theta(n)$, where $n = r - p + 1$ is the total number of elements being merged, and it works as follows. Returning to our card-playing motif, suppose we have two piles of cards face up on a table. Each pile is sorted, with the smallest cards on top. We wish to merge the two piles into a single sorted output pile, which is to be face down on the table. Our basic step consists of choosing the smaller of the two cards on top of the face-up piles, removing it from its pile (which exposes a new top card), and placing this card face down onto

the output pile. We repeat this step until one input pile is empty, at which time we just take the remaining input pile and place it face down onto the output pile. Computationally, each basic step takes constant time, since we are comparing just the two top cards. Since we perform at most n basic steps, merging takes $\Theta(n)$ time.

The following pseudocode implements the above idea, but with an additional twist that avoids having to check whether either pile is empty in each basic step. We place on the bottom of each pile a ***sentinel*** card, which contains a special value that we use to simplify our code. Here, we use ∞ as the sentinel value, so that whenever a card with ∞ is exposed, it cannot be the smaller card unless both piles have their sentinel cards exposed. But once that happens, all the nonsentinel cards have already been placed onto the output pile. Since we know in advance that exactly $r - p + 1$ cards will be placed onto the output pile, we can stop once we have performed that many basic steps.

```
MERGE(A, p, q, r)
 1  n_1 = q − p + 1
 2  n_2 = r − q
 3  let L[1 . . n_1 + 1] and R[1 . . n_2 + 1] be new arrays
 4  for i = 1 to n_1
 5      L[i] = A[p + i − 1]
 6  for j = 1 to n_2
 7      R[j] = A[q + j]
 8  L[n_1 + 1] = ∞
 9  R[n_2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```

In detail, the MERGE procedure works as follows. Line 1 computes the length n_1 of the subarray $A[p \,..\, q]$, and line 2 computes the length n_2 of the subarray $A[q + 1 \,..\, r]$. We create arrays L and R ("left" and "right"), of lengths $n_1 + 1$ and $n_2 + 1$, respectively, in line 3; the extra position in each array will hold the sentinel. The **for** loop of lines 4–5 copies the subarray $A[p \,..\, q]$ into $L[1 \,..\, n_1]$, and the **for** loop of lines 6–7 copies the subarray $A[q + 1 \,..\, r]$ into $R[1 \,..\, n_2]$. Lines 8–9 put the sentinels at the ends of the arrays L and R. Lines 10–17, illus-

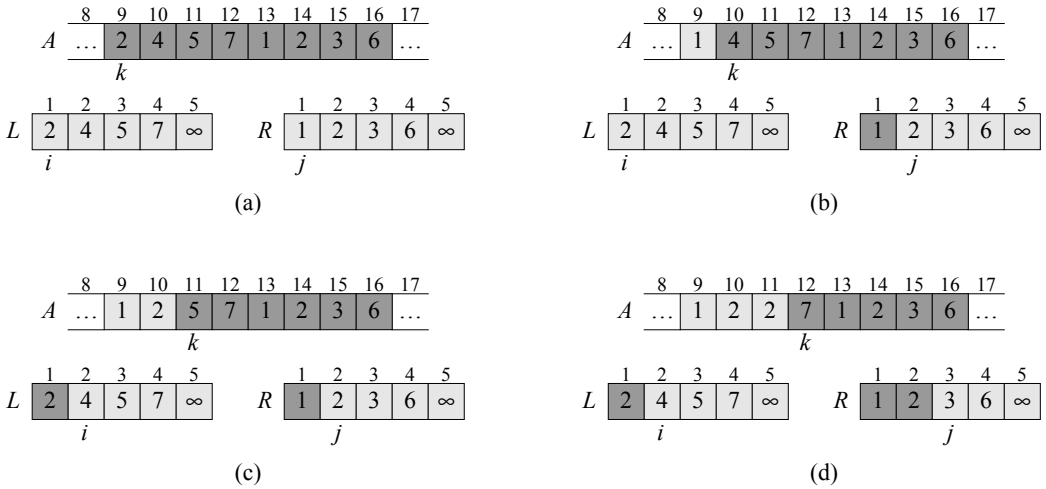


Figure 2.3 The operation of lines 10–17 in the call MERGE$(A, 9, 12, 16)$, when the subarray $A[9..16]$ contains the sequence $\langle 2, 4, 5, 7, 1, 2, 3, 6\rangle$. After copying and inserting sentinels, the array L contains $\langle 2, 4, 5, 7, \infty\rangle$, and the array R contains $\langle 1, 2, 3, 6, \infty\rangle$. Lightly shaded positions in A contain their final values, and lightly shaded positions in L and R contain values that have yet to be copied back into A. Taken together, the lightly shaded positions always comprise the values originally in $A[9..16]$, along with the two sentinels. Heavily shaded positions in A contain values that will be copied over, and heavily shaded positions in L and R contain values that have already been copied back into A. **(a)–(h)** The arrays A, L, and R, and their respective indices k, i, and j prior to each iteration of the loop of lines 12–17.

trated in Figure 2.3, perform the $r - p + 1$ basic steps by maintaining the following loop invariant:

> At the start of each iteration of the **for** loop of lines 12–17, the subarray $A[p..k-1]$ contains the $k - p$ smallest elements of $L[1..n_1 + 1]$ and $R[1..n_2 + 1]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A.

We must show that this loop invariant holds prior to the first iteration of the **for** loop of lines 12–17, that each iteration of the loop maintains the invariant, and that the invariant provides a useful property to show correctness when the loop terminates.

Initialization: Prior to the first iteration of the loop, we have $k = p$, so that the subarray $A[p..k-1]$ is empty. This empty subarray contains the $k - p = 0$ smallest elements of L and R, and since $i = j = 1$, both $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A.

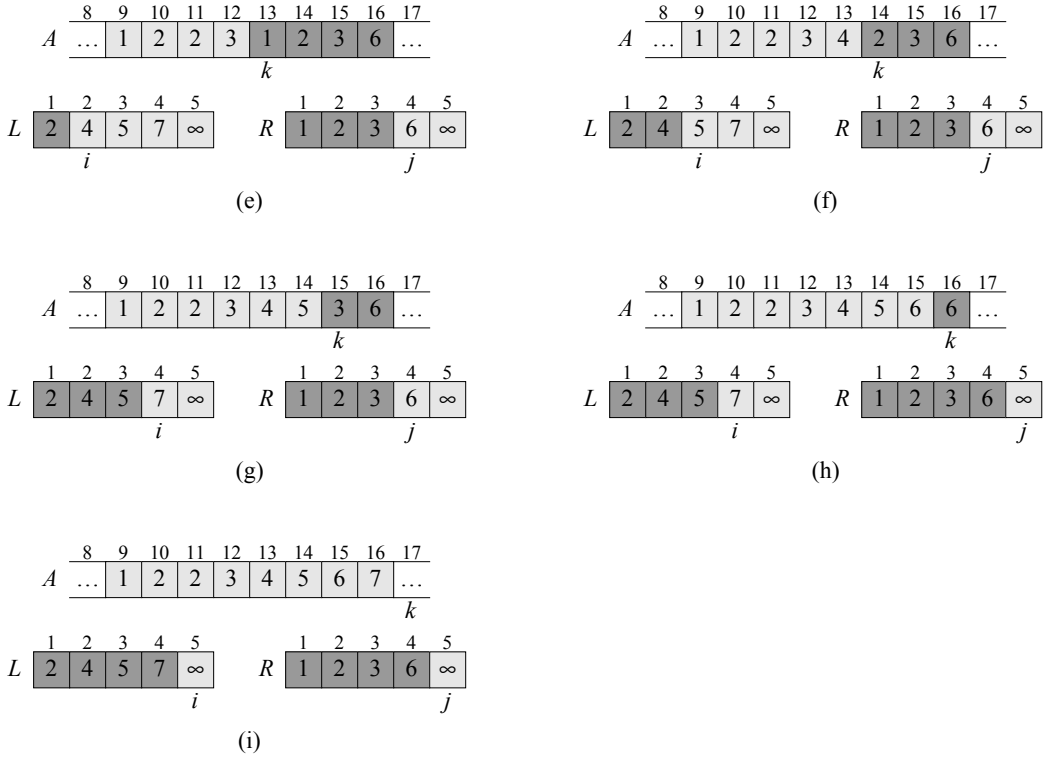


Figure 2.3, continued **(i)** The arrays and indices at termination. At this point, the subarray in $A[9\,..\,16]$ is sorted, and the two sentinels in L and R are the only two elements in these arrays that have not been copied into A.

Maintenance: To see that each iteration maintains the loop invariant, let us first suppose that $L[i] \le R[j]$. Then $L[i]$ is the smallest element not yet copied back into A. Because $A[p\,..\,k-1]$ contains the $k-p$ smallest elements, after line 14 copies $L[i]$ into $A[k]$, the subarray $A[p\,..\,k]$ will contain the $k-p+1$ smallest elements. Incrementing k (in the **for** loop update) and i (in line 15) reestablishes the loop invariant for the next iteration. If instead $L[i] > R[j]$, then lines 16–17 perform the appropriate action to maintain the loop invariant.

Termination: At termination, $k = r+1$. By the loop invariant, the subarray $A[p\,..\,k-1]$, which is $A[p\,..\,r]$, contains the $k-p = r-p+1$ smallest elements of $L[1\,..\,n_1+1]$ and $R[1\,..\,n_2+1]$, in sorted order. The arrays L and R together contain $n_1+n_2+2 = r-p+3$ elements. All but the two largest have been copied back into A, and these two largest elements are the sentinels.

To see that the MERGE procedure runs in $\Theta(n)$ time, where $n = r - p + 1$, observe that each of lines 1–3 and 8–11 takes constant time, the **for** loops of lines 4–7 take $\Theta(n_1 + n_2) = \Theta(n)$ time,[7] and there are n iterations of the **for** loop of lines 12–17, each of which takes constant time.

We can now use the MERGE procedure as a subroutine in the merge sort algorithm. The procedure MERGE-SORT(A, p, r) sorts the elements in the subarray $A[p..r]$. If $p \geq r$, the subarray has at most one element and is therefore already sorted. Otherwise, the divide step simply computes an index q that partitions $A[p..r]$ into two subarrays: $A[p..q]$, containing $\lceil n/2 \rceil$ elements, and $A[q+1..r]$, containing $\lfloor n/2 \rfloor$ elements.[8]

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

To sort the entire sequence $A = \langle A[1], A[2], \ldots, A[n] \rangle$, we make the initial call MERGE-SORT$(A, 1, A.length)$, where once again $A.length = n$. Figure 2.4 illustrates the operation of the procedure bottom-up when n is a power of 2. The algorithm consists of merging pairs of 1-item sequences to form sorted sequences of length 2, merging pairs of sequences of length 2 to form sorted sequences of length 4, and so on, until two sequences of length $n/2$ are merged to form the final sorted sequence of length n.

2.3.2 Analyzing divide-and-conquer algorithms

When an algorithm contains a recursive call to itself, we can often describe its running time by a ***recurrence equation*** or ***recurrence***, which describes the overall running time on a problem of size n in terms of the running time on smaller inputs. We can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.

---

[7] We shall see in Chapter 3 how to formally interpret equations containing Θ-notation.

[8] The expression $\lceil x \rceil$ denotes the least integer greater than or equal to x, and $\lfloor x \rfloor$ denotes the greatest integer less than or equal to x. These notations are defined in Chapter 3. The easiest way to verify that setting q to $\lfloor (p + r)/2 \rfloor$ yields subarrays $A[p..q]$ and $A[q+1..r]$ of sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$, respectively, is to examine the four cases that arise depending on whether each of p and r is odd or even.

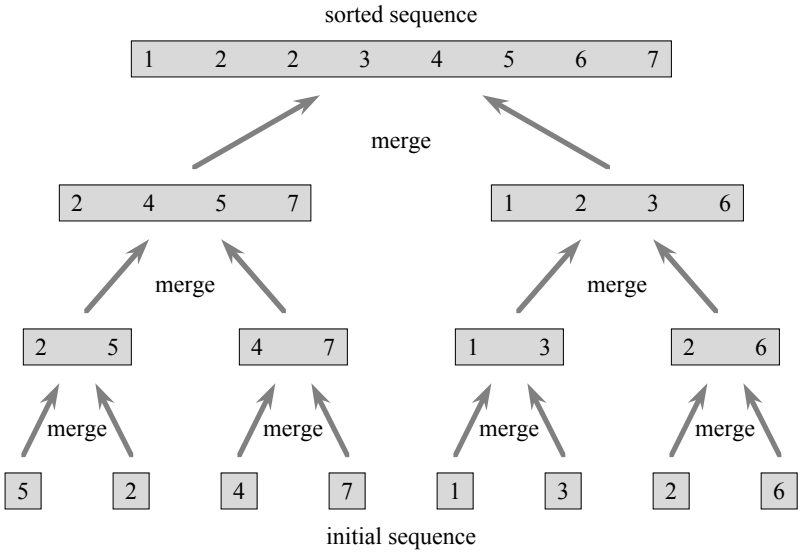


Figure 2.4 The operation of merge sort on the array $A = \langle 5, 2, 4, 7, 1, 3, 2, 6 \rangle$. The lengths of the sorted sequences being merged increase as the algorithm progresses from bottom to top.

A recurrence for the running time of a divide-and-conquer algorithm falls out from the three steps of the basic paradigm. As before, we let $T(n)$ be the running time on a problem of size n. If the problem size is small enough, say $n \leq c$ for some constant c, the straightforward solution takes constant time, which we write as $\Theta(1)$. Suppose that our division of the problem yields a subproblems, each of which is $1/b$ the size of the original. (For merge sort, both a and b are 2, but we shall see many divide-and-conquer algorithms in which $a \neq b$.) It takes time $T(n/b)$ to solve one subproblem of size n/b, and so it takes time $aT(n/b)$ to solve a of them. If we take $D(n)$ time to divide the problem into subproblems and $C(n)$ time to combine the solutions to the subproblems into the solution to the original problem, we get the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \ , \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \ . \end{cases}$$

In Chapter 4, we shall see how to solve common recurrences of this form.

Analysis of merge sort

Although the pseudocode for MERGE-SORT works correctly when the number of elements is not even, our recurrence-based analysis is simplified if we assume that

the original problem size is a power of 2. Each divide step then yields two subsequences of size exactly $n/2$. In Chapter 4, we shall see that this assumption does not affect the order of growth of the solution to the recurrence.

We reason as follows to set up the recurrence for $T(n)$, the worst-case running time of merge sort on n numbers. Merge sort on just one element takes constant time. When we have $n > 1$ elements, we break down the running time as follows.

Divide: The divide step just computes the middle of the subarray, which takes constant time. Thus, $D(n) = \Theta(1)$.

Conquer: We recursively solve two subproblems, each of size $n/2$, which contributes $2T(n/2)$ to the running time.

Combine: We have already noted that the MERGE procedure on an n-element subarray takes time $\Theta(n)$, and so $C(n) = \Theta(n)$.

When we add the functions $D(n)$ and $C(n)$ for the merge sort analysis, we are adding a function that is $\Theta(n)$ and a function that is $\Theta(1)$. This sum is a linear function of n, that is, $\Theta(n)$. Adding it to the $2T(n/2)$ term from the "conquer" step gives the recurrence for the worst-case running time $T(n)$ of merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 , \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 . \end{cases} \tag{2.1}$$

In Chapter 4, we shall see the "master theorem," which we can use to show that $T(n)$ is $\Theta(n \lg n)$, where $\lg n$ stands for $\log_2 n$. Because the logarithm function grows more slowly than any linear function, for large enough inputs, merge sort, with its $\Theta(n \lg n)$ running time, outperforms insertion sort, whose running time is $\Theta(n^2)$, in the worst case.

We do not need the master theorem to intuitively understand why the solution to the recurrence (2.1) is $T(n) = \Theta(n \lg n)$. Let us rewrite recurrence (2.1) as

$$T(n) = \begin{cases} c & \text{if } n = 1 , \\ 2T(n/2) + cn & \text{if } n > 1 , \end{cases} \tag{2.2}$$

where the constant c represents the time required to solve problems of size 1 as well as the time per array element of the divide and combine steps.[9]

---

[9] It is unlikely that the same constant exactly represents both the time to solve problems of size 1 and the time per array element of the divide and combine steps. We can get around this problem by letting c be the larger of these times and understanding that our recurrence gives an upper bound on the running time, or by letting c be the lesser of these times and understanding that our recurrence gives a lower bound on the running time. Both bounds are on the order of $n \lg n$ and, taken together, give a $\Theta(n \lg n)$ running time.

Figure 2.5 shows how we can solve recurrence (2.2). For convenience, we assume that n is an exact power of 2. Part (a) of the figure shows $T(n)$, which we expand in part (b) into an equivalent tree representing the recurrence. The cn term is the root (the cost incurred at the top level of recursion), and the two subtrees of the root are the two smaller recurrences $T(n/2)$. Part (c) shows this process carried one step further by expanding $T(n/2)$. The cost incurred at each of the two subnodes at the second level of recursion is $cn/2$. We continue expanding each node in the tree by breaking it into its constituent parts as determined by the recurrence, until the problem sizes get down to 1, each with a cost of c. Part (d) shows the resulting ***recursion tree***.

Next, we add the costs across each level of the tree. The top level has total cost cn, the next level down has total cost $c(n/2) + c(n/2) = cn$, the level after that has total cost $c(n/4) + c(n/4) + c(n/4) + c(n/4) = cn$, and so on. In general, the level i below the top has 2^i nodes, each contributing a cost of $c(n/2^i)$, so that the ith level below the top has total cost $2^i\, c(n/2^i) = cn$. The bottom level has n nodes, each contributing a cost of c, for a total cost of cn.

The total number of levels of the recursion tree in Figure 2.5 is $\lg n + 1$, where n is the number of leaves, corresponding to the input size. An informal inductive argument justifies this claim. The base case occurs when $n = 1$, in which case the tree has only one level. Since $\lg 1 = 0$, we have that $\lg n + 1$ gives the correct number of levels. Now assume as an inductive hypothesis that the number of levels of a recursion tree with 2^i leaves is $\lg 2^i + 1 = i + 1$ (since for any value of i, we have that $\lg 2^i = i$). Because we are assuming that the input size is a power of 2, the next input size to consider is 2^{i+1}. A tree with $n = 2^{i+1}$ leaves has one more level than a tree with 2^i leaves, and so the total number of levels is $(i + 1) + 1 = \lg 2^{i+1} + 1$.

To compute the total cost represented by the recurrence (2.2), we simply add up the costs of all the levels. The recursion tree has $\lg n + 1$ levels, each costing cn, for a total cost of $cn(\lg n + 1) = cn \lg n + cn$. Ignoring the low-order term and the constant c gives the desired result of $\Theta(n \lg n)$.

Exercises

2.3-1

Using Figure 2.4 as a model, illustrate the operation of merge sort on the array $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

2.3-2

Rewrite the MERGE procedure so that it does not use sentinels, instead stopping once either array L or R has had all its elements copied back to A and then copying the remainder of the other array back into A.

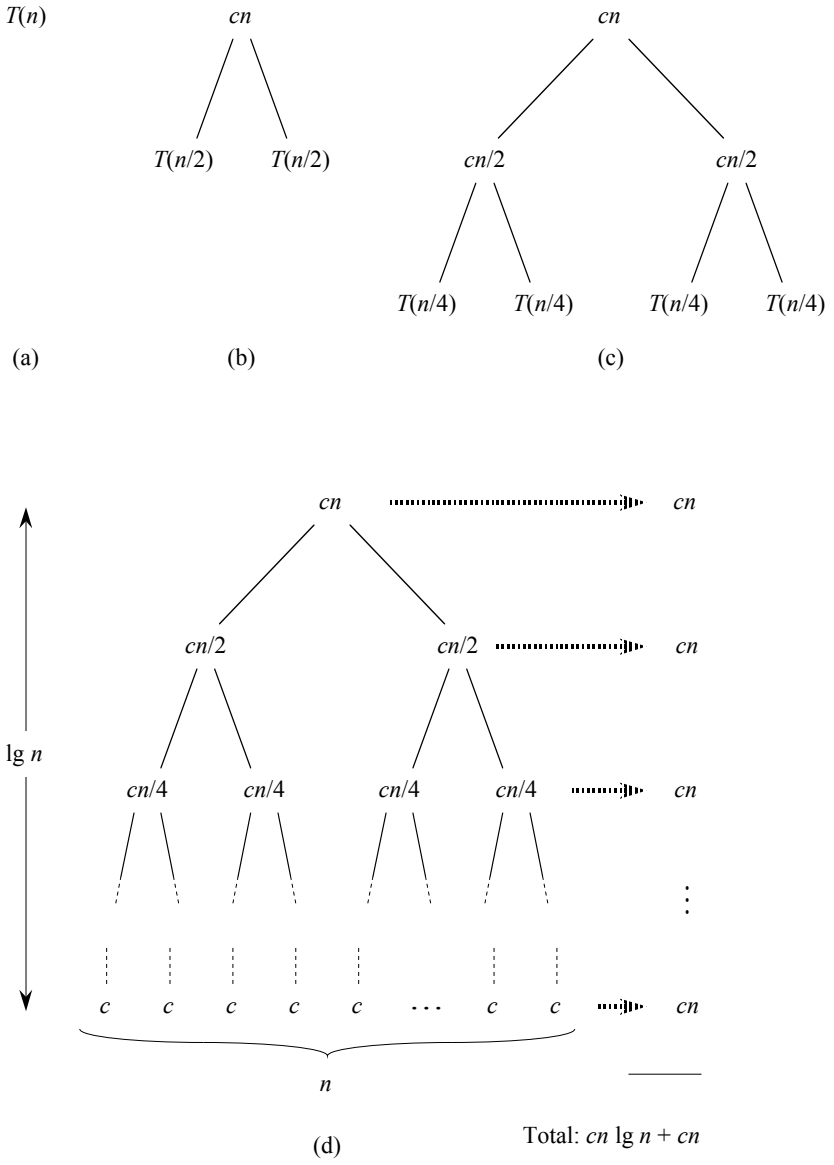


Figure 2.5 How to construct a recursion tree for the recurrence $T(n) = 2T(n/2) + cn$. Part **(a)** shows $T(n)$, which progressively expands in **(b)–(d)** to form the recursion tree. The fully expanded tree in part (d) has $\lg n + 1$ levels (i.e., it has height $\lg n$, as indicated), and each level contributes a total cost of cn. The total cost, therefore, is $cn \lg n + cn$, which is $\Theta(n \lg n)$.

2.3-3
Use mathematical induction to show that when n is an exact power of 2, the solution of the recurrence

$$T(n) = \begin{cases} 2 & \text{if } n = 2\,, \\ 2T(n/2) + n & \text{if } n = 2^k, \text{ for } k > 1 \end{cases}$$

is $T(n) = n \lg n$.

2.3-4
We can express insertion sort as a recursive procedure as follows. In order to sort $A[1\,..\,n]$, we recursively sort $A[1\,..\,n-1]$ and then insert $A[n]$ into the sorted array $A[1\,..\,n-1]$. Write a recurrence for the worst-case running time of this recursive version of insertion sort.

2.3-5
Referring back to the searching problem (see Exercise 2.1-3), observe that if the sequence A is sorted, we can check the midpoint of the sequence against v and eliminate half of the sequence from further consideration. The ***binary search*** algorithm repeats this procedure, halving the size of the remaining portion of the sequence each time. Write pseudocode, either iterative or recursive, for binary search. Argue that the worst-case running time of binary search is $\Theta(\lg n)$.

2.3-6
Observe that the **while** loop of lines 5–7 of the INSERTION-SORT procedure in Section 2.1 uses a linear search to scan (backward) through the sorted subarray $A[1\,..\,j-1]$. Can we use a binary search (see Exercise 2.3-5) instead to improve the overall worst-case running time of insertion sort to $\Theta(n \lg n)$?

2.3-7 ★
Describe a $\Theta(n \lg n)$-time algorithm that, given a set S of n integers and another integer x, determines whether or not there exist two elements in S whose sum is exactly x.

Problems

2-1 Insertion sort on small arrays in merge sort
Although merge sort runs in $\Theta(n \lg n)$ worst-case time and insertion sort runs in $\Theta(n^2)$ worst-case time, the constant factors in insertion sort can make it faster in practice for small problem sizes on many machines. Thus, it makes sense to ***coarsen*** the leaves of the recursion by using insertion sort within merge sort when

b. What array with elements from the set $\{1, 2, \ldots, n\}$ has the most inversions? How many does it have?

c. What is the relationship between the running time of insertion sort and the number of inversions in the input array? Justify your answer.

d. Give an algorithm that determines the number of inversions in any permutation on n elements in $\Theta(n \lg n)$ worst-case time. (*Hint:* Modify merge sort.)

Chapter notes

In 1968, Knuth published the first of three volumes with the general title *The Art of Computer Programming* [209, 210, 211]. The first volume ushered in the modern study of computer algorithms with a focus on the analysis of running time, and the full series remains an engaging and worthwhile reference for many of the topics presented here. According to Knuth, the word "algorithm" is derived from the name "al-Khowârizmî," a ninth-century Persian mathematician.

Aho, Hopcroft, and Ullman [5] advocated the asymptotic analysis of algorithms—using notations that Chapter 3 introduces, including Θ-notation—as a means of comparing relative performance. They also popularized the use of recurrence relations to describe the running times of recursive algorithms.

Knuth [211] provides an encyclopedic treatment of many sorting algorithms. His comparison of sorting algorithms (page 381) includes exact step-counting analyses, like the one we performed here for insertion sort. Knuth's discussion of insertion sort encompasses several variations of the algorithm. The most important of these is Shell's sort, introduced by D. L. Shell, which uses insertion sort on periodic subsequences of the input to produce a faster sorting algorithm.

Merge sort is also described by Knuth. He mentions that a mechanical collator capable of merging two decks of punched cards in a single pass was invented in 1938. J. von Neumann, one of the pioneers of computer science, apparently wrote a program for merge sort on the EDVAC computer in 1945.

The early history of proving programs correct is described by Gries [153], who credits P. Naur with the first article in this field. Gries attributes loop invariants to R. W. Floyd. The textbook by Mitchell [256] describes more recent progress in proving programs correct.

S5.1 Merge sort: ranges, recursion, and recombination

Divide, conquer, and combine

Merge sort splits an input range into two smaller ranges, recursively sorts both, and merges the ordered results. The split does not itself sort records. The combine step relies on both recursive results already being ordered. A range containing zero or one record is the base case and needs no rearrangement.

Half-open ranges

For zero-based indexing, use [lo,hi), which contains hi-lo records and excludes hi. Let mid=lo+(hi-lo)//2. The children are [lo,mid) and [mid,hi); they do not overlap, and their union is the parent range. Stop when hi-lo<=1.

The CLRS presentation uses one-based inclusive ranges. Its midpoint and recursion bounds are correct under that convention. Translate the whole range contract rather than copying only one endpoint formula into a half-open implementation.

A complete split pattern

For [5,1,4,2,8], split into [5,1] and [4,2,8]. The first child becomes [1,5]. The second splits into [4] and [2,8], then merges into [2,4,8]. Merging the two children yields [1,2,4,5,8]. Odd lengths are normal: the two child sizes differ by at most one.

Why recursion terminates

When the parent size exceeds one, both child sizes are smaller than the parent size. Repeated halving eventually reaches a singleton. A missing base case or a midpoint that reproduces the parent interval can cause infinite recursion.

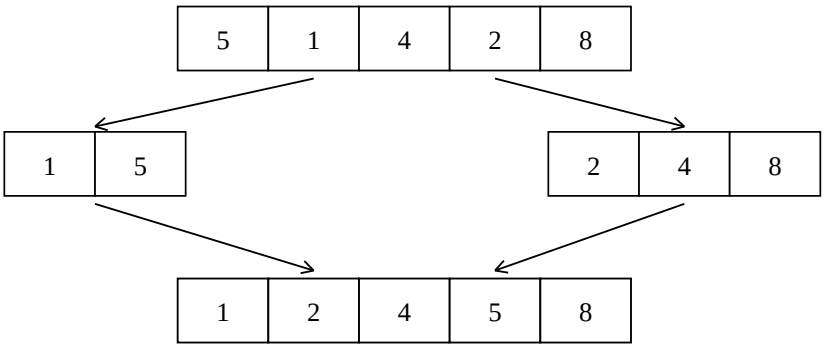


Figure S5.1 Unequal child lengths still partition and sort the entire input.

New explanatory material; original source: CLRS 2.3.1; original supplementary explanation

S5.2 The merge operation: a full two-pointer trace

The smallest available head

Let L=[1,4,7] and R=[2,4,6]. Initially both read pointers are zero and the output is empty. Compare the heads and emit 1 from L. Then compare 4 and 2 and emit 2 from R. Compare the two 4s and emit the left 4 first for stability. Compare 7 and the right 4 and emit the right 4. Compare 7 and 6 and emit 6.

R is now exhausted. Append the remaining 7 from L without a key comparison. The output is [1,2,4,4,6,7]. There were five key comparisons and six output writes. The key-comparison count for nonempty runs of sizes a and b is at most a+b-1; copying the unconsumed remainder supplies the rest of the writes.

Invariant for the output prefix

Before each comparison, the output contains exactly the already consumed records from L and R in ordered form, and those records are no greater than every unconsumed record. Because each input run is ordered, its head is its smallest unconsumed key. Choosing the smaller head therefore selects a smallest remaining record overall.

Advance only the pointer for the run from which a record was emitted. Forgetting to advance repeats a record and can prevent termination; advancing both can skip a record. When one run is exhausted, the remaining records in the other run are already ordered and no smaller than the output boundary.

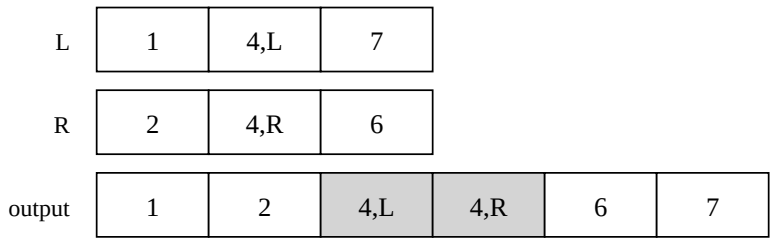


Figure S5.2 Left-first equality preserves the tie order of the two runs.

New explanatory material; original source: CLRS 2.3.1; original worked example

S5.3 Merge sort: time recurrences and memory accounting

A recurrence with uneven splits

For n>1, conventional merge sort satisfies T(n)=T(floor(n/2))+T(ceil(n/2))+Theta(n). The Theta(n) combine term includes writing or copying all records in the parent range. For powers of two this simplifies to T(n)=2T(n/2)+Theta(n), with a constant-time singleton base case.

Each nonleaf recursion level handles a total of n records. The tree has Theta(log n) levels, giving Theta(n log n) total time for the conventional always-merge implementation. The input's existing order does not remove the required copying work in this version. An optimized implementation that checks the two-child boundary and skips an unnecessary merge can have a different best case.

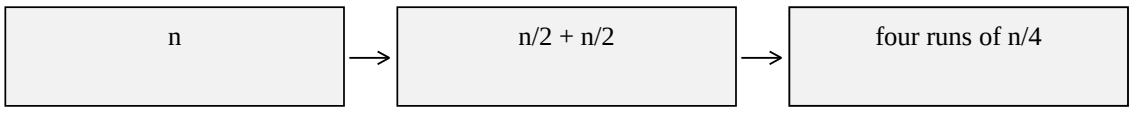
Peak auxiliary space

A reusable temporary array of size n needs O(n) records. The recursion stack has O(log n) frames for balanced splitting. Together these are O(n) auxiliary space. If temporary child arrays are created by slicing, account for their actual lifetimes; balanced depth-first recursion can still have O(n) peak live element storage, while generating O(n log n) total allocation traffic over the execution.

Do not add storage across time as though it were all live simultaneously. Conversely, retaining every temporary version or every recursion-level snapshot for visualization can increase memory and is a different implementation from a normal merge sort.

Comparison lower bound

A comparison decision tree for n distinct keys must distinguish n! orderings, so worst-case height is at least log2(n!)=Omega(n log n). Merge sort matches this order of growth in the comparison model. Counting sort uses value indexing and therefore operates outside that lower-bound model.



Volume per level: n Levels: Theta(log n)

Combine volume over all levels: Theta(n log n)

Figure S5.3 Total combine volume is linear at each logarithmic recursion level.

New explanatory material; original source: CLRS 2.3.2; original supplementary derivation

S5.4 Merge sort: a stable sentinel-free implementation

A fresh-output version

This listing returns a new list rather than sorting the caller's list in place. The recursive inputs are slices; stability follows from stable recursion and left-first equality during each merge.

```
def merge_sort(a, key=lambda x: x):
    if len(a) <= 1:
        return list(a)
    mid = len(a)//2
    left = merge_sort(a[:mid], key)
    right = merge_sort(a[mid:], key)
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out
```

Sentinels are a separate design

CLRS presents a merge using an infinity sentinel. A practical sentinel must be greater than every permitted input key and compatible with the comparison domain. An ordinary maximum integer is unsafe if that same value is a valid data key. A unique sentinel object also needs an appropriate comparator. The listing above avoids those issues by testing whether either run is exhausted.

Proof obligations and checks

The split ranges partition the original sequence, recursive results preserve their records, and merge emits every record exactly once. The smaller-head invariant proves order. On equal keys, left-first selection preserves records from the original left range before equal records from the right; combined with stable child sorts, this gives global stability.

Check [], a singleton, odd lengths, duplicated labeled keys across both halves, and maximum permitted keys. Do not use a mutation-based test expectation for this returning-a-new-list contract.

S5.5 Merge sort: stability, descending order, and adaptivity

Stability requires consistent tie handling

Every record in the original left half precedes every record in the original right half. When equal keys are at both heads, choosing the left record preserves that relationship. If the child sorts are stable and every merge uses this rule, an induction on recursion size proves the entire sort stable.

For descending order, choose the left head when its key is greater than or equal to the right head. Equal records still come from the left first. Reversing a stable ascending output would reverse tie groups and is therefore not a substitute for stable descending merging.

Skip a merge only when justified

After sorting both children, if the maximum left key is no greater than the minimum right key, their concatenation is already ordered. With ascending ranges, check the last left record against the first right record. Skipping the merge preserves stability because the left group's equal records already precede the right group's.

For an implementation using indexed ranges and a reusable buffer, this boundary test can yield Theta(n) best-case work on already ordered input: the recursion visits O(n) nodes and each successful skip costs constant time. A version that still slices or copies whole sublists at every level may retain Theta(n log n) copying cost even if it skips comparisons. State the implementation before claiming linear best time.

Natural runs and bottom-up merging

Bottom-up merge sort combines runs of size 1, then 2, then 4, and so on, avoiding a recursive stack while still needing an array merge buffer. A natural merge variant starts from detected ordered runs and can exploit existing structure. These are distinct variants; do not apply the standard recurrence blindly to every adaptive run strategy.

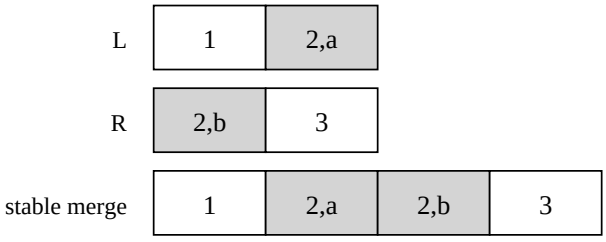


Figure S5.5 A safe skip checks the ordered child boundary after both children are sorted.

New explanatory material; original source: CLRS 2.3.1; original supplementary variants

S5.6 Merge sort: inversion counting as a derived application

Count pairs across the split

Each inversion belongs either entirely to the left range, entirely to the right range, or crosses the split. Recursive calls count the first two classes. During an ascending merge, if right[j] is strictly smaller than left[i], every remaining left record is greater than right[j], because the left run is ordered. Add len(left)-i to the cross-inversion count when emitting that right record.

If left[i] is no greater than right[j], emit the left record without adding a crossing inversion. Equality must not be counted as a strict inversion. The total count is left_count + right_count + cross_count. This procedure has Theta(n log n) running time using the standard merge recurrence.

A worked merge

For left=[2,4,5] and right=[1,3], emitting 1 from the right adds 3 crossing inversions: (2,1), (4,1), and (5,1). Emit 2 from the left. Then emitting 3 from the right adds 2: (4,3) and (5,3). The cross count is 5, and the merged output is [1,2,3,4,5].

The original array [2,4,5,1,3] has no inversions internal to either already ordered half, so its full count is 5. If the halves were not originally ordered, their internal inversion counts must be retained from the recursive calls rather than ignored.

Use the result carefully

The count measures pair disorder. It helps explain insertion-sort shifts and strict adjacent bubble exchanges. It does not equal the number of selection-sort swaps, and it is not the number of positions occupied by the wrong value.

For very large n, n(n-1)/2 can exceed a fixed-width integer's maximum. Choose a count representation large enough for the contract. Python integers grow as needed, but their arithmetic cost is not literally constant for arbitrarily large counts.

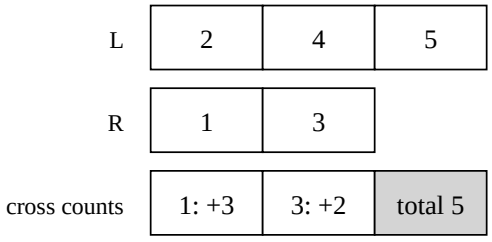


Figure S5.6 One smaller right head accounts for several crossing inversions.

New explanatory material; original source: CLRS Problem 2-4; original worked derivation

S5.7 Merge sort: deep questions and practical boundaries

Can merge sort be in place?

The conventional array merge routine discussed here uses a linear auxiliary buffer and is not in place under the O(1)-extra-storage definition. More sophisticated in-place merging methods exist, but they have different implementations and tradeoffs. A statement about the standard method should not claim that every possible merge sort necessarily requires the same buffer.

Linked-list merging can rearrange links rather than copy a linear array buffer, since it can repeatedly take a run's head. Recursion frames, run bookkeeping, and output representation still need accounting. "No array buffer" is not automatically "zero auxiliary space".

Why does merging help external sorting?

Ordered runs can be read sequentially and combined into another ordered stream. When records do not fit memory, one can create memory-sized sorted runs and then merge them with bounded input/output buffers. Disk I/O, number of runs, merge fan-in, and buffer sizes become important cost parameters. The in-memory Theta(n log n) CPU bound alone does not specify an external-sorting I/O bound.

Parallel work is not automatic speedup

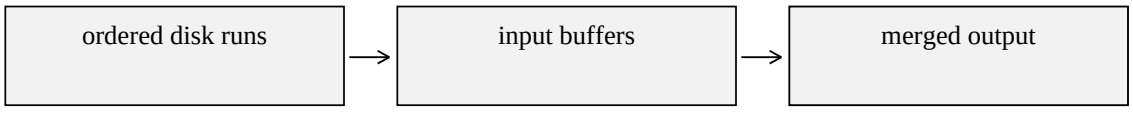
The two child sorts concern disjoint ranges and can be performed independently. A sequential merge still consumes time proportional to the parent range. Actual parallel speed depends on task overhead, memory bandwidth, available workers, and whether merging is parallelized; recursion alone does not establish a numeric speedup.

Question bank

"Why is merge sort better than insertion for large disordered input?" Its worst-case growth is n log n rather than quadratic, under the stated model. "Why might insertion win on a small input?" Lower overhead and few shifts can outweigh the asymptotic distinction. "Why doesn't counting sort violate merge's optimality?" Optimality applies within the comparison model, while counting uses a restricted integer domain and value-indexed storage.

Answering a variant question

State whether the routine is top-down or bottom-up, array or list based, stable or right-first on ties, buffered or using a special in-place merge, and adaptive or always merging. These choices control both guarantees and trace interpretation.



Read sequentially; account for buffers and I/O separately.

Figure S5.7 Ordered runs can be merged sequentially with explicit buffers.

New explanatory material; original source: CLRS 2.3 and 8.1; original supplementary synthesis

8.1-2
Obtain asymptotically tight bounds on $\lg(n!)$ without using Stirling's approximation. Instead, evaluate the summation $\sum_{k=1}^{n} \lg k$ using techniques from Section A.2.

8.1-3
Show that there is no comparison sort whose running time is linear for at least half of the $n!$ inputs of length n. What about a fraction of $1/n$ of the inputs of length n? What about a fraction $1/2^n$?

8.1-4
Suppose that you are given a sequence of n elements to sort. The input sequence consists of n/k subsequences, each containing k elements. The elements in a given subsequence are all smaller than the elements in the succeeding subsequence and larger than the elements in the preceding subsequence. Thus, all that is needed to sort the whole sequence of length n is to sort the k elements in each of the n/k subsequences. Show an $\Omega(n \lg k)$ lower bound on the number of comparisons needed to solve this variant of the sorting problem. (*Hint:* It is not rigorous to simply combine the lower bounds for the individual subsequences.)

8.2 Counting sort

Counting sort assumes that each of the n input elements is an integer in the range 0 to k, for some integer k. When $k = O(n)$, the sort runs in $\Theta(n)$ time.

Counting sort determines, for each input element x, the number of elements less than x. It uses this information to place element x directly into its position in the output array. For example, if 17 elements are less than x, then x belongs in output position 18. We must modify this scheme slightly to handle the situation in which several elements have the same value, since we do not want to put them all in the same position.

In the code for counting sort, we assume that the input is an array $A[1..n]$, and thus $A.length = n$. We require two other arrays: the array $B[1..n]$ holds the sorted output, and the array $C[0..k]$ provides temporary working storage.

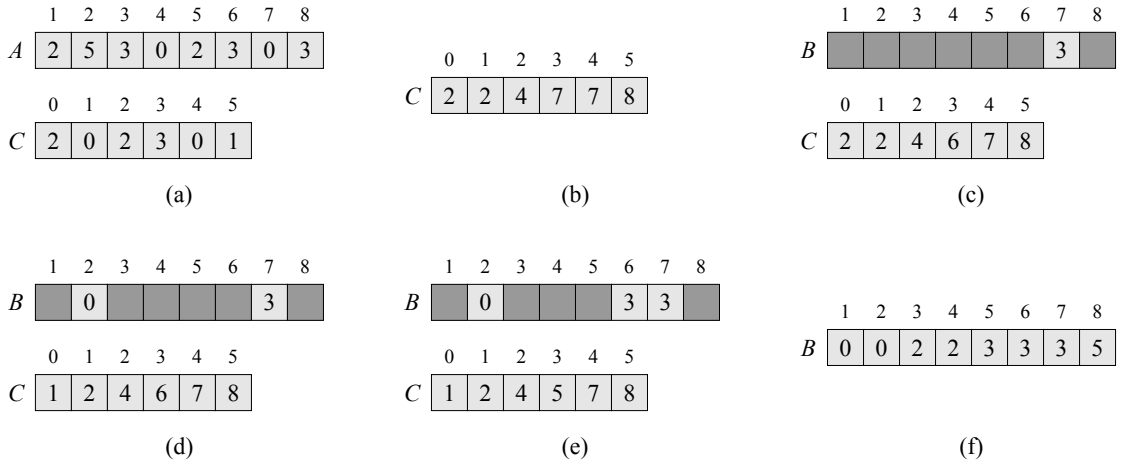


Figure 8.2 The operation of COUNTING-SORT on an input array $A[1..8]$, where each element of A is a nonnegative integer no larger than $k = 5$. **(a)** The array A and the auxiliary array C after line 5. **(b)** The array C after line 8. **(c)–(e)** The output array B and the auxiliary array C after one, two, and three iterations of the loop in lines 10–12, respectively. Only the lightly shaded elements of array B have been filled in. **(f)** The final sorted output array B.

```
COUNTING-SORT(A, B, k)
 1  let C[0..k] be a new array
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to A.length
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  for j = A.length downto 1
11      B[C[A[j]]] = A[j]
12      C[A[j]] = C[A[j]] − 1
```

Figure 8.2 illustrates counting sort. After the **for** loop of lines 2–3 initializes the array C to all zeros, the **for** loop of lines 4–5 inspects each input element. If the value of an input element is i, we increment $C[i]$. Thus, after line 5, $C[i]$ holds the number of input elements equal to i for each integer $i = 0, 1, \ldots, k$. Lines 7–8 determine for each $i = 0, 1, \ldots, k$ how many input elements are less than or equal to i by keeping a running sum of the array C.

Finally, the **for** loop of lines 10–12 places each element $A[j]$ into its correct sorted position in the output array B. If all n elements are distinct, then when we first enter line 10, for each $A[j]$, the value $C[A[j]]$ is the correct final position of $A[j]$ in the output array, since there are $C[A[j]]$ elements less than or equal to $A[j]$. Because the elements might not be distinct, we decrement $C[A[j]]$ each time we place a value $A[j]$ into the B array. Decrementing $C[A[j]]$ causes the next input element with a value equal to $A[j]$, if one exists, to go to the position immediately before $A[j]$ in the output array.

How much time does counting sort require? The **for** loop of lines 2–3 takes time $\Theta(k)$, the **for** loop of lines 4–5 takes time $\Theta(n)$, the **for** loop of lines 7–8 takes time $\Theta(k)$, and the **for** loop of lines 10–12 takes time $\Theta(n)$. Thus, the overall time is $\Theta(k + n)$. In practice, we usually use counting sort when we have $k = O(n)$, in which case the running time is $\Theta(n)$.

Counting sort beats the lower bound of $\Omega(n \lg n)$ proved in Section 8.1 because it is not a comparison sort. In fact, no comparisons between input elements occur anywhere in the code. Instead, counting sort uses the actual values of the elements to index into an array. The $\Omega(n \lg n)$ lower bound for sorting does not apply when we depart from the comparison sort model.

An important property of counting sort is that it is ***stable***: numbers with the same value appear in the output array in the same order as they do in the input array. That is, it breaks ties between two numbers by the rule that whichever number appears first in the input array appears first in the output array. Normally, the property of stability is important only when satellite data are carried around with the element being sorted. Counting sort's stability is important for another reason: counting sort is often used as a subroutine in radix sort. As we shall see in the next section, in order for radix sort to work correctly, counting sort must be stable.

Exercises

8.2-1

Using Figure 8.2 as a model, illustrate the operation of COUNTING-SORT on the array $A = \langle 6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 \rangle$.

8.2-2

Prove that COUNTING-SORT is stable.

8.2-3

Suppose that we were to rewrite the **for** loop header in line 10 of the COUNTING-SORT as

10 **for** $j = 1$ **to** $A.length$

Show that the algorithm still works properly. Is the modified algorithm stable?

8.2-4
Describe an algorithm that, given n integers in the range 0 to k, preprocesses its input and then answers any query about how many of the n integers fall into a range $[a\,..\,b]$ in $O(1)$ time. Your algorithm should use $\Theta(n + k)$ preprocessing time.

8.3 Radix sort

Radix sort is the algorithm used by the card-sorting machines you now find only in computer museums. The cards have 80 columns, and in each column a machine can punch a hole in one of 12 places. The sorter can be mechanically "programmed" to examine a given column of each card in a deck and distribute the card into one of 12 bins depending on which place has been punched. An operator can then gather the cards bin by bin, so that cards with the first place punched are on top of cards with the second place punched, and so on.

For decimal digits, each column uses only 10 places. (The other two places are reserved for encoding nonnumeric characters.) A d-digit number would then occupy a field of d columns. Since the card sorter can look at only one column at a time, the problem of sorting n cards on a d-digit number requires a sorting algorithm.

Intuitively, you might sort numbers on their *most significant* digit, sort each of the resulting bins recursively, and then combine the decks in order. Unfortunately, since the cards in 9 of the 10 bins must be put aside to sort each of the bins, this procedure generates many intermediate piles of cards that you would have to keep track of. (See Exercise 8.3-5.)

Radix sort solves the problem of card sorting—counterintuitively—by sorting on the *least significant* digit first. The algorithm then combines the cards into a single deck, with the cards in the 0 bin preceding the cards in the 1 bin preceding the cards in the 2 bin, and so on. Then it sorts the entire deck again on the second-least significant digit and recombines the deck in a like manner. The process continues until the cards have been sorted on all d digits. Remarkably, at that point the cards are fully sorted on the d-digit number. Thus, only d passes through the deck are required to sort. Figure 8.3 shows how radix sort operates on a "deck" of seven 3-digit numbers.

In order for radix sort to work correctly, the digit sorts must be stable. The sort performed by a card sorter is stable, but the operator has to be wary about not changing the order of the cards as they come out of a bin, even though all the cards in a bin have the same digit in the chosen column.

S6.1 Counting sort: domain and range are part of the input

Integer keys with a manageable range

Counting sort exploits the fact that keys can index a finite array. For keys 0 through k inclusive, the range has R=k+1 values. It counts occurrences, computes cumulative positions, and distributes records directly into key-ordered output positions. It does not determine order by pairwise comparisons of input records.

The inclusive range matters: keys 0 through 5 require six counters, not five. In the stable output-array form, time is Theta(n+R) and auxiliary space is O(n+R). Calling this linear in n requires R=O(n), or a fixed range under the chosen analysis. With ten keys whose maximum is a billion, the range array is usually an inappropriate direct representation.

Two forms with different promises

A frequency-only numeric sort counts values and writes each value as many times as its frequency. That can reconstruct sorted numeric values using O(R) counters when overwriting the input, but it does not by itself preserve identities of records carrying satellite data.

Stable record counting sort uses an output array and a placement rule that preserves equal-key arrival order. The algorithm explained in CLRS belongs to this form. Do not attach its stability guarantee to a numeric frequency reconstruction routine unless the record handling has been specified.

Mapping other domains

Negative integers can be offset by their minimum value. A finite categorical ordering can be encoded as integer ranks if that rank order is explicitly defined. Arbitrary strings or floating-point values cannot simply be used as count-array indices. A rank-compression preprocessing step may require comparison sorting and must be counted separately.

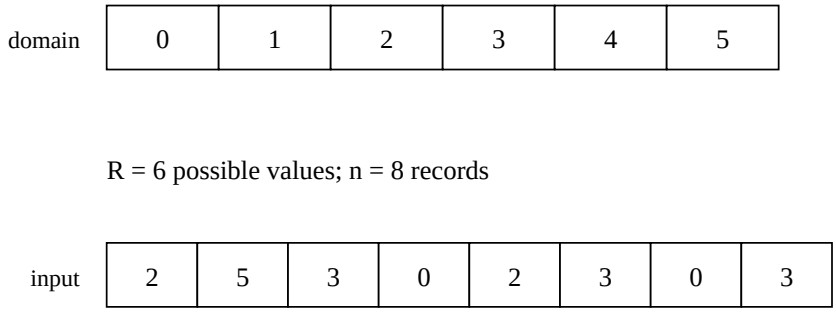


Figure S6.1 The number of available key values differs from the number of records.

New explanatory material; original source: CLRS 8.2; original supplementary explanation

S6.2 Counting sort: histogram, cumulative counts, and positions

Histogram phase

Use A=[2,5,3,0,2,3,0,3], with keys 0 through 5. Initial frequencies are F=[2,0,2,3,0,1]. The entry F[v] is the number of records whose key equals v, and the sum of all frequency entries is n=8.

Cumulative phase

Compute C[v]=F[0]+...+F[v], giving C=[2,2,4,7,7,8]. C[v] is the number of keys no greater than v. It is nondecreasing, not a frequency histogram. In one-based output positions, key v ends at position C[v]. In a zero-based output list, its current last available position is C[v]-1.

For key 3, seven records have keys <=3. Its final group occupies zero-based positions 4 through 6, since four keys are strictly smaller and three keys equal 3. Distinguish that interval from the scalar C[3]=7.

Placement phase

Traverse A from right to left. The last 3 goes to B[6], then decrement C[3] from 7 to 6. The preceding 0 goes to B[1], decrementing C[0] from 2 to 1. The preceding 3 goes to B[5], and later the earliest 3 goes to B[4]. After all records are placed, B=[0,0,2,2,3,3,3,5].

The C array is consumed as a next-free-position structure. Its final values no longer represent the original cumulative counts. If later range queries require the cumulative totals, preserve a separate copy before placement.

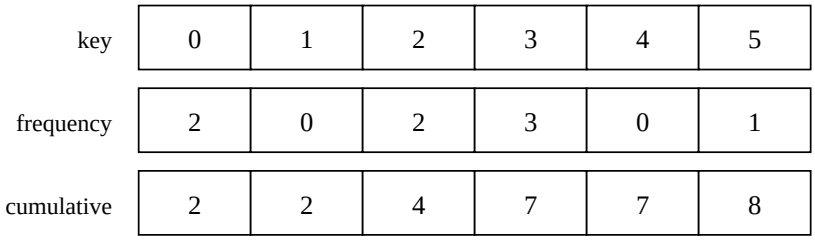


Figure S6.2 Frequencies become end positions; end positions are then consumed.

New explanatory material; original source: CLRS 8.2; original worked example

S6.3 Counting sort: stable placement and correctness

Disjoint reserved intervals

Let P[v] be the original cumulative number of keys <=v and let P[-1]=0 for the proof. Key v owns the zero-based interval [P[v-1],P[v]). Its length is the frequency of v. Different keys have disjoint intervals, and smaller keys own earlier intervals.

Initially C[v]=P[v]. Each placement of a key-v record writes to C[v]-1 and decrements C[v]. Thus it fills the reserved interval from right to left without collision. Exactly the key's frequency many records are processed, so its interval is filled exactly once. Together the intervals fill all n output positions and preserve every record.

Why reverse input traversal is stable

Suppose equal-key record x precedes y in the input. A right-to-left traversal processes y first, placing it in a later available slot. When x is processed, the counter has moved left, so x receives an earlier slot. Their output order is therefore the same as their input order.

If the same cumulative-end-position rule processes input left to right, x receives the later slot and y the earlier slot. Numeric keys remain correctly sorted, but equal-key record identities are reversed within their group. This answers the distinction in Exercise 8.2-3.

A different stable forward variant

Forward input traversal can still be stable if it uses starting positions rather than ending positions. Initialize next[v] to the number of keys strictly smaller than v, write to B[next[v]], then increment next[v]. The first equal record gets the first slot. Direction alone is not the proof; direction and position-update rule must agree.

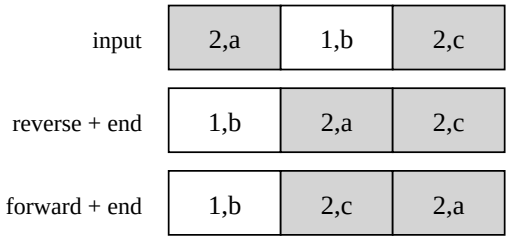


Figure S6.3 End positions pair with reverse traversal; start positions pair with forward traversal.

New explanatory material; original source: CLRS 8.2 and Exercises 8.2-2, 8.2-3; original proof

S6.4 Counting sort: signed integers, sparse ranges, and space

Offset the key domain

For signed integer keys, let lo be the minimum and hi the maximum. Allocate R=hi-lo+1 counters and map a key x to index x-lo. The transformation preserves numeric order: x<y if and only if x-lo<y-lo. Store full original records in the output, not merely the mapped indices.

For A=[-2,1,-2,0], lo=-2 and hi=1. The mapped indices are [0,3,0,2] and the frequencies are [2,0,1,1]. Output records have keys [-2,-2,0,1]. Handle the empty list before requesting a minimum or maximum.

Range size controls feasibility

Finding min and max costs Theta(n) and does not remove the O(R) initialization or cumulative scan. A large gap between lo and hi creates counters for absent keys too. For keys [-1000000,1000000], only two values occur but the direct range has 2000001 entries.

A hash-map histogram stores only observed keys, but producing globally ordered key groups still requires ordering those keys or a separately known domain order. If m observed distinct keys are comparison-sorted, that step costs O(m log m). It is not justified to call such a modified method unconditional linear-time counting sort.

Space distinctions

The stable record version has O(n+R) extra storage for output and counts. A numeric frequency-reconstruction version may overwrite the input and use O(R) counters, but it does not preserve arbitrary satellite records. If a caller supplies a reusable output buffer, report whether that buffer is included in the stated auxiliary-space measure.

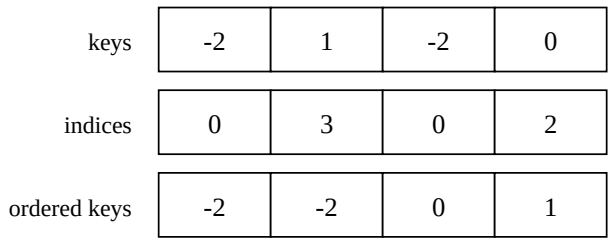


Figure S6.4 Offsetting signed keys preserves order but does not shrink a large gap.

New explanatory material; original source: CLRS 8.2; original supplementary adaptations

S6.5 Counting sort: a stable record implementation

Contract and implementation

This function returns a new list, accepts negative integer keys, and preserves input order among equal keys. Its range allocation must fit the application's memory budget. This teaching implementation assumes every extracted key is an integer; a production input validator should enforce the stated domain.

```
def counting_sort(a, key=lambda x: x):
    if not a:
        return []
    keys = [key(x) for x in a]
    lo, hi = min(keys), max(keys)
    counts = [0] * (hi-lo+1)
    for k in keys:
        counts[k-lo] += 1
    for v in range(1, len(counts)):
        counts[v] += counts[v-1]
    out = [None] * len(a)
    for j in range(len(a)-1, -1, -1):
        v = keys[j]-lo
        counts[v] -= 1
        out[counts[v]] = a[j]
    return out
```

Decrementing immediately before the write is equivalent to writing at the old cumulative count minus one and then decrementing. It is not an arbitrary off-by-one change. All array accesses use zero-based positions and offset key indices.

Checks and cost

For [(2,'a'),(-1,'b'),(2,'c')], the first-field key gives [(-1,'b'),(2,'a'),(2,'c')]. The original list remains in its original order. Every out slot must be filled; an unfilled None would expose a placement or domain error.

Caching extracted keys costs O(n) additional storage beyond counts and output, but the total remains O(n+R). The extraction, min/max, counting, and output passes are linear in n; initialization and cumulative counting are linear in R. Empty input bypasses min/max and range allocation.

S6.6 Counting sort: range queries and radix-sort connections

Preprocess inclusive range counts

Let P[v] be the number of keys <=v before output placement consumes any counters. Define count_le(x)=0 below the minimum allowed key and count_le(x)=n at or above the maximum. Within the range, look up the corresponding cumulative entry.

The number of input keys in the inclusive interval [a,b] is count_le(b)-count_le(a-1) for integer a<=b. If a>b, return zero by the query contract. Computing a histogram and its cumulative array takes Theta(n+R); each subsequent valid interval query takes O(1) under machine-integer indexing assumptions.

For [2,5,3,0,2,3,0,3], there are five values in [2,3]: P[3]-P[1]=7-2=5. There are two values in [0,1], and zero in [4,4]. The inclusive subtraction boundary a-1 is crucial; P[b]-P[a] would incorrectly exclude records equal to a.

Why a digit pass needs stability

Least-significant-digit radix sorting first orders the lowest digit, then successively more significant digits. Each pass must preserve the order already established among records sharing the current digit. A stable counting sort can be the digit-pass subroutine when the digit alphabet is a small integer range.

For two-digit keys [21,11,12], the units pass produces [21,11,12], retaining 21 before 11 for the shared units digit 1. A stable tens pass produces [11,12,21], preserving the units order within tens group 1. An unstable tens pass could instead place 12 before 11 and lose the earlier work.

The full radix bound depends on digit count d and digit range R: a straightforward implementation costs Theta(d(n+R)). The counting-sort bound alone does not make arbitrary-width integer radix sorting linear without those parameters.

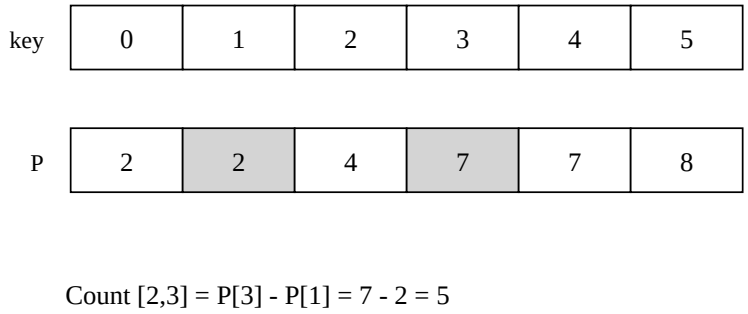


Figure S6.6 Cumulative counts answer intervals; stable digit passes retain prior order.

New explanatory material; original source: CLRS Exercises 8.2-4 and 8.3-2; original supplementary explanation

S6.7 Counting sort: deep questions, limits, and corpus audit

Does linear counting sort contradict a lower bound?

No. The Omega(n log n) theorem applies to worst-case comparison sorting of arbitrary distinct comparable keys, where ordering information comes only from pairwise comparisons. Counting sort assumes bounded integer keys and uses their values to address an array. Those additional operations and domain restrictions change the model.

Is it always the fastest of these five methods?

No. Theta(n+R) can be dominated by a huge R. Memory allocation, zero-initialization, payload handling, key extraction, and the actual comparison cost all affect a concrete execution. A domain and range check is necessary before choosing it. An asymptotic formula is not a measured performance ranking.

Can it be stable and in place simultaneously?

The conventional CLRS stable record version uses an output array and is not O(1)-extra-space. Numeric reconstruction may overwrite input values with O(R) counters, but that is a different contract and cannot preserve record identities by value reconstruction alone. Specialized variants require their own algorithm and proof rather than borrowing the CLRS properties wholesale.

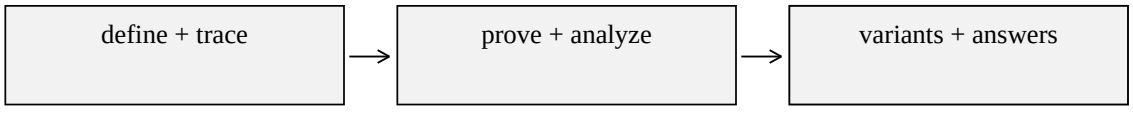
Coverage audit of this combined corpus

Principles now include contracts, identities, ties, cost models, bounds, invariants, special domains, semantic interpretation, and method choice. Bubble and selection include complete explanations, pseudocode, traces, proofs, exact counts, stability variants, implementations, and answer banks. Insertion adds inversion-sensitive analysis, binary insertion, and hybrids. Merge adds sentinel-free code, range conventions, peak-space accounting, adaptive variants, and inversion counting. Counting adds signed domains, stable placement proofs, range queries, and radix connections.

The 55 retained source pages remain unchanged. A replacement contents page and 44 supplements produce exactly 100 PDF pages. Source references retain printed pagination; supplements use S-section names and PDF positions.

Remaining boundaries

This focused corpus covers the five requested methods. Brief adjoining topics remain on preserved whole source pages. The checked teaching code illustrates its stated contracts; application-specific validation remains separate.



55 preserved source pages + 44 supplements + 1 contents = 100

Figure S6.7 Each requested method now has definitions, mechanisms, guarantees, and conditioned answers.

New explanatory material; original source: CLRS 8.1-8.2; original supplementary synthesis